

Privacy-Preserving Agentic RAG for Healthcare and Humanitarian Intelligence *

Master's Thesis

Jérémy Chaverot
Student
LiGHT
EPFL
jeremy.chaverot@epfl.ch

Fabrice Nemo
Supervision
LiGHT
EPFL
fabrice.nemo@epfl.ch

David Sasu
Supervision
LiGHT
EPFL
david.sasu@epfl.ch

Mary-Anne Hartley
Professor
LiGHT
EPFL
mary-anne.hartley@epfl.ch

July 3rd, 2026



Citation: **Jérémy Chaverot**; **Fabrice Nemo and David Sasu** (Supervisors); **Mary-Anne Hartley** (Professor). *Privacy-Preserving Agentic RAG for Healthcare and Humanitarian Intelligence* . July 2026 . Master's Thesis. Laboratory for Intelligent Global Health and Humanitarian Response Technologies (LiGHT), EPFL.

Abstract

BACKGROUND.

Retrieval-Augmented Generation lets a language model ground its answers in a set of documents. These systems are valuable, but they often operate on highly sensitive documents, particularly in healthcare and humanitarian domains. A standard pipeline passes the retrieved text unchanged into the answer model prompt, so confidential information travels from the corpus to a model that cannot always be trusted. Existing solutions sanitize the text once and assume the result is correct, or tie protection to keeping the model on local hardware, without leveraging the reasoning capabilities of frontier models. It remains unclear whether decomposing privacy into cooperating components, with an adversarial step that checks for leakage before the external model is queried and for hallucinations after, can prevent disclosure while preserving answer quality.

METHODS.

This work is a system-design contribution evaluated under a fixed comparison protocol. A domain-aware multi-agent privacy layer is added to an existing RAG framework (MMORE) and operates between retrieval and generation: it selects a domain profile, removes or replaces identifying information, inspects the sanitized context before it reaches an untrusted answer model, and checks the returned answer for faithfulness to the retrieved evidence. It is evaluated on open synthetic clinical corpora, comparing alternative detection and sanitization choices against a no-privacy baseline and a single-pass sanitization baseline.

RESULTS.

The privacy layer is tested on an open medical dataset, measuring leakage to the external model and answer quality against both baselines, and isolating the effect of the adversarial gate. For the humanitarian domain, it is applied in CHITCHAT, a project whose Humani-T toolkit guides humanitarian organizations in developing and evaluating language-model systems aligned with humanitarian principles, drawing on their own confidential documents.

CONCLUSION.

This work shows that privacy protection in retrieval-augmented generation can be split into cooperating, auditable components, so that the privacy guarantee no longer depends on running the answer model locally. This lets organizations use strong external models on confidential documents without exposing raw content. The next step is validation in a live humanitarian deployment.

Keywords privacy · data anonymization · retrieval-augmented generation · multi-agent systems · adversarial verification · healthcare · humanitarian domain

1 Introduction

1.1 Background and Research Gap

Retrieval-augmented generation (RAG) lets a large language model answer a question using relevant passages retrieved from a document collection rather than from its parametric memory alone [1]. This makes RAG attractive for clinical question answering over patient records and for the analysis of humanitarian field reports about affected populations. In both settings the source documents carry sensitive personal information: patient names, medical record numbers, or admission dates in the first case; locations, ethnicity, or legal status in the second, to name a few. A standard RAG pipeline does not transform this text. Retrieved passages are copied directly into the prompt of the answer model, so *personally identifiable information* (PII) travels unchanged from the corpus to the model. When the answer model is a hosted commercial service, the retrieved passages leave the data owner's trust boundary on every query.

This exposure is the dominant barrier to deploying RAG over real clinical and humanitarian corpora, more so than answer quality. A recent review of RAG-enhanced healthcare chatbots surveys deployed applications and privacy-preserving strategies and formalizes a three-stage threat model covering data storage, data transmission, and data retrieval and generation [2]. Beyond the practical risk, protecting this data is a concrete legal requirement [3–5]. Above all, the stakes are human: in healthcare, a disclosure erodes the confidence patients place in their health institutions, and in the humanitarian domain it can put the very lives of the people receiving aid at risk.

Prior work on privacy-preserving RAG follows a few lines. Static sanitization and knowledge erasure remove identifiers before indexing or generation [6, 7]. Cryptographic and decoding-time defenses perturb embeddings, queries, or the output distribution [8, 9]. Differential privacy adds calibrated noise to the embeddings or retrieval mechanisms [10]. Synthetic-corpus methods replace the sensitive collection with generated surrogates [11], and on-device anonymisation sanitizes before querying an external model [12].

In these solutions, the privacy problem is treated as a one-pass step. It therefore remains unknown whether decomposing privacy into cooperating specialized components, with an adversarial component that probes the sanitized context before the answer model is queried and the generated answer after, can prevent personal information from reaching an untrusted model while preserving retrieval and answer quality. It is also unknown whether such a design transfers across domains that differ in what counts as sensitive.

This work addresses the gap by introducing a domain-aware, multi-agent privacy layer that operates between retrieval and generation in an existing RAG framework, and by evaluating it on healthcare and humanitarian corpora.

1.2 Research Aim and Objectives

The aim of this work is to design and evaluate a multi-agent privacy layer for retrieval-augmented generation that sanitizes retrieved context and adversarially verifies, before and after generation, that no personal information reaches an untrusted answer model, while preserving retrieval and answer quality across the healthcare and humanitarian domains.

The work is structured around three research questions.

RQ1. How can a multi-agent system meet professionals' expectations for accurate, responsive, and trustworthy retrieval-augmented generation while preserving patient privacy?

RQ2. Which combinations of detection engine and sanitization strategy protect privacy without degrading answer quality?

RQ3. To what extent does adversarial verification reduce privacy leakage?

The aim is met through the following objectives.

1. To review existing privacy-preserving RAG approaches and position this work within the healthcare RAG privacy taxonomy, making explicit which paradigm is adopted and which gap is addressed (Section 2).
2. To design and implement the multi-agent privacy layer, comprising context and policy analysis, PII detection, sanitization, and double verification, as an orchestrated graph integrated into the MMORE RAG framework, addressing RQ1 (Section 4).
3. To compare PII detection engines and sanitization strategies and quantify their effect on retrieval and answer quality, addressing RQ2 (Sections 4 and 5).
4. To evaluate the extent to which adversarial verification reduces information leakage relative to single-pass sanitization, addressing RQ3 (Section 5).

5. To demonstrate the system's cross-domain design beyond healthcare through its direct application to the humanitarian domain in the CHITCHAT project, addressing RQ1 (Section 6.3).

1.3 Contributions

This work makes the following contributions.

- **End-to-end privacy layer.** A modular and fully configurable multi-agent privacy layer for RAG, comprising a context and policy analyzer, a detector, a sanitizer, a pre-generation leakage adversary, and a post-generation verifier, orchestrated as a state graph and integrated seamlessly into the MMORE RAG framework.
- **Adversarial verification.** A pre-generation leakage gate that triggers a bounded re-sanitization loop before any context leaves the trust boundary, and a post-generation verifier that checks the answer for residual leakage and for faithfulness to the retrieved evidence.
- **Engine and strategy comparison.** A comparative evaluation of four PII detection engines (Microsoft Presidio, GLiNER-PII, `openai/privacy-filter`, and an LLM detector) and four sanitization strategies (token masking, entity replacement, Presidio anonymization, and synthetic rewrite), reporting their effect on retrieval and answer quality.
- **Privacy–utility evaluation.** An evaluation of the privacy and utility trade-off, and of information leakage with and without adversarial verification, on an healthcare corpora. For the humanitarian domain, we have a direct application thanks to the CHITCHAT project.

This work establishes a privacy-preserving RAG architecture and its evaluation protocol for healthcare and humanitarian document collections, addressing only the data retrieval and generation stage of the threat model. It does not claim protection against storage-stage or transmission-stage attackers, which are handled by the encryption and access controls inherited from the surrounding deployment, and it does not claim clinical deployment readiness.

Alongside the privacy layer, this work includes a set of independent engineering contributions to the MMORE framework, carried out with a software engineering approach of problem analysis, design trade-off, implementation, and validation. These tasks are distinct from the main research contribution and are documented in Appendix C.

2 Related Work

This work sits at the intersection of three lines of research, and this section reviews each in turn. We first cover *retrieval-augmented generation* (Section 2.1), which supplies the grounding pipeline this work extends but treats retrieved context as trusted plaintext. We then survey *privacy in RAG and large language models* (Section 2.2), which establishes the leakage threat and a family of single-pass defenses. We finally review *agentic harnesses and multi-agent engineering* (Section 2.3), which provides the orchestration, reflection, and evaluation primitives this work repurposes for privacy enforcement. Along the way, we point out where each strand falls short, before drawing them together in Section 2.4.

2.1 Retrieval-augmented generation

Retrieval-augmented generation was introduced in May 2020 by Patrick Lewis and colleagues [13]. It lets a parametric language model condition its output on passages fetched from a non-parametric store, improving factuality on knowledge-intensive tasks. A parallel line integrates retrieval directly into pre-training so that the model learns to attend to retrieved documents [14]. Both foundational designs share a property that is harmless for open-domain question answering but consequential here: the retrieved text is copied word for word into the model's context with no notion of whether it is sensitive, so any identifier present in the documents can propagate into the prompt and leak. This is precisely the hole that a query-time privacy layer can close.

Subsequent work has enriched the pipeline along the retrieval and control axes rather than the privacy axis. Surveys document a broad design space of chunking, indexing, re-ranking, and fusion strategies for RAG [1]. Asai et al. [15] add self-reflection so the model decides when to retrieve and critiques its own generations against the retrieved evidence, which is notable here because it shows that a retrieval pipeline can host reflective control loops, although the critique targets answer quality rather than disclosure. In the clinical setting, Xiong et al. [16] benchmark RAG across medical corpora and retrievers, and Wada et al. [17] report that on-premise RAG raises local model quality in radiology consultation at practical latency, confirming the utility of grounded retrieval over sensitive corpora while leaving the protection of those corpora out of scope.

RAG is now very good at grounding answers, and most of its design effort has gone into retrieval quality. What it does not yet include is a step that inspects the retrieved passages and cleans them before they reach the model. This is a natural place to add one, provided it works at query time and preserves the retrieval quality the pipeline already delivers.

The MMORE framework. This work builds on MMORE, the open-source Massive Multimodal Open RAG and Extraction pipeline [18] (Figure 2.1). MMORE was developed in our own lab (LiGHT at EPFL, Switzerland), and its code is publicly available. This makes it a natural choice here: because we control every step of the pipeline, we can seamlessly insert a modular privacy layer without treating any part as a black box. MMORE ingests more than fifteen heterogeneous file types, including text, tables, images, email, audio, and video, and turns them into a markdown representation. It then indexes the result with hybrid dense-sparse retrieval. It can be used in two ways: a batch mode that answers a whole list of questions at once, and an interactive mode that answers questions one by one in real time. MMORE can also run as a server with a RESTful API, so a client can upload documents or retrieve relevant chunks. All pipelines are CPU-friendly, but can also leverage GPU parallelism for efficiency. It therefore supplies precisely the backbone this work extends. Notably, its authors identify processing in privacy-sensitive settings as future work; this is exactly the gap the privacy layer developed here fills.

2.2 Privacy in RAG and large language models

The leakage risk is well established. Carlini et al. [19] showed that large language models memorize and can regurgitate training data verbatim, and Kim et al. [20] built probes that surface personal information from a deployed model. Zeng et al. [21] extended this diagnosis to RAG specifically, demonstrating that data from documents used for retrieval can be extracted through crafted queries. These foundational studies establish the threat and quantify it, but they are largely measurement and attack papers: they motivate a defense without providing a deployable one. This matters because protection is also a legal requirement: the HIPAA Safe Harbor standard requires 18 categories of identifier to be removed before data counts as de-identified [3]; GDPR Article 53 places health data in a special category that may be processed only on an explicit legal basis [4]; and the EU AI Act classifies clinical decision support as high-risk, requiring human oversight and transparency [5]. Two recent works consolidate the threat surface: Guan et al. [2] review healthcare RAG chatbots under a three-stage threat model of storage, transmission, and retrieval-and-generation, and Arzanipour et al. [22] formalize the RAG threat model and attack surface.

Deployable defenses fall into a few families. Static sanitization and knowledge erasure remove identifiers before or during retrieval [6, 7]. Differential privacy adds calibrated noise to the retrieval-generation process [10]. Guan

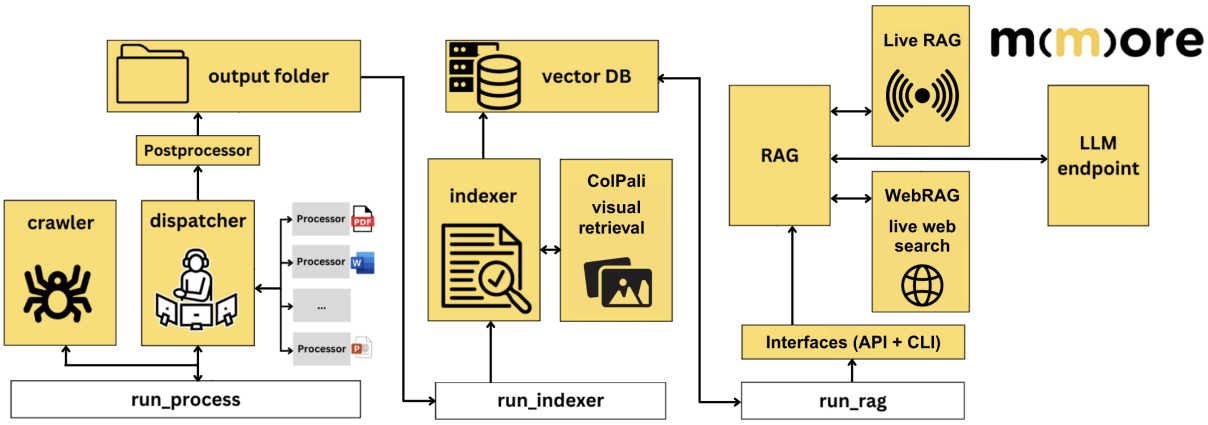


Figure 2.1: The MMORE pipeline. Heterogeneous input files are ingested and converted to a unified markdown representation, post-processed and chunked, embedded, and indexed for hybrid dense-sparse retrieval, then served in batch, interactive, or API mode. This work inserts the privacy layer at query time, between retrieval and generation.

et al. [2] note that in clinical use this noise can shift a value across a decision threshold while remaining invisible to text-similarity metrics. Synthetic-corpus methods replace the sensitive collection with generated surrogates [11], trading fidelity for safety. Cryptographic, split-inference, and federated defenses keep computation off a single trusted host [8, 23, 24], and decoding-time defenses perturb the output distribution [9]. Both families tie protection to a specific protocol or to where computation runs. Closest to this work, Weerasekara et al. [12] anonymise protected health information on-device before querying an external model, Fang et al. [25] substitute sensitive entities with symbolic placeholders recovered locally after generation, and Velzen et al. [26] use generative agents to synthesize privacy-preserving clinical documentation.

Each of these families cleans the text once and trusts the result, and many tie their guarantee to a particular protocol or to keeping the model on local hardware. What is still missing is a way to check that the protection actually worked, a need that Guan et al. [2] highlight when they call for automated privacy-assessment tools. This opens the door to a defense that shares the work across cooperating parts, checks outputs before and after the model is queried, and no longer depends only on where the answer model runs.

2.3 Agentic harnesses and multi-agent engineering

The engineering primitives this work builds on come from the agentic-LLM literature. Yao et al. [27] interleave reasoning traces with tool actions so a model can plan and act iteratively, and Shinn et al. [28] add verbal self-reflection, letting an agent critique a failed attempt and retry with that feedback. These foundations are directly relevant because a bounded re-sanitization loop is a reflection loop. In both cases, however, the self-critique optimizes task success rather than disclosure, and a single agent grading its own output is a known weak point, which motivates an independent checker.

Beyond these single-agent foundations, our design draws on three further ideas. Multi-agent frameworks decompose a task across specialized, communicating agents [29, 30], which is the pattern this work applies to the detect-sanitize-verify pipeline. Programmatic prompt optimization compiles declarative agent specifications into tuned prompts [31], the approach used for the prompted detector here. The evaluation primitive is LLM-as-a-judge, in which one model scores another's output [32]: the same mechanism, turned adversarial, underlies the leakage gate and the post-generation verifier. Finally, Singh et al. [33] survey *agentic RAG*, in which agents mediate retrieval and generation, and confirm that this composition is largely applied to answer quality and tool use rather than to privacy enforcement.

Reflection, multi-agent decomposition, and LLM-as-a-judge evaluation are well proven for reasoning and answer quality, but they have rarely been used for privacy. The same ideas fit naturally here: an agent that actively looks for leaks, and a loop that keeps cleaning the context until it is safe, built into the RAG pipeline as a core step rather than added on at the end.

2.4 Synthesis and positioning

These three research directions are complementary and, taken together, expose the gap this work addresses. RAG delivers grounded answers but passes retrieved context to the model as trusted plaintext. The privacy literature confirms that this context leaks. Existing defenses sanitize the text once or tie protection to a specific protocol or to

local hardware, and rarely check afterwards that the protection held. Agentic engineering contributes reflection, decomposition, and LLM-as-a-judge evaluation, but applies them to answer quality rather than to disclosure. This work combines the three research areas in a domain-aware multi-agent layer that treats privacy as the task. The layer applies agentic decomposition to detection and sanitization, and adds adversarial verification, in the spirit of LLM-as-a-judge, both before and after generation. The privacy guarantee is thus decoupled from where the answer model runs. Section 4 details the resulting architecture.

3 Domain Expert Requirements

The system serves two domains, healthcare and humanitarian. They do not agree on what counts as sensitive, and the harm caused by a disclosure is different in each. The literature alone was therefore not enough, so we also asked people who work in these fields what they need. For healthcare, we ran a short structured survey of clinicians. For the humanitarian side, we relied on one in-depth expert interview. This section reports the two separately. What we learned shapes the two domain presets, the ready-made settings the system loads for each field, and the threat model described in Section 4.

3.1 Requirements-gathering method

For healthcare, we sent a short structured survey to clinicians and received four responses: an experienced infectious-disease physician working for a humanitarian organization, an orthopaedic-surgery resident at a regional hospital, an internal-medicine clinician-researcher at a university hospital, and a general practitioner in private practice. The survey asked about their current practice, how they access and share data, what they would use the tool for, their daily workflow, what earns their trust, and how much error they can tolerate. The full questionnaire is reproduced in Appendix B. As the sample is small, we use it to choose sensible default settings and to build realistic test scenarios, not to draw statistical conclusions.

The humanitarian sector is broad. It spans medical care, infrastructure, legal advice, and international human-rights work. A broad but shallow survey would have missed this depth, so we held one detailed interview instead. Our expert is a senior specialist in digital transformation at the International Committee of the Red Cross (ICRC), and knows the organization's data-protection policy well.

3.2 Healthcare clinician survey

3.2.1 Findings

Today, searching a patient record is slow and manual. The information is scattered across separate systems: the main clinical record, the laboratory system, the imaging archive (often called a PACS, the hospital system that stores medical images), and even scanned paper. There is no reliable way to search or filter, and the dates on documents cannot always be trusted. What clinicians need most is a clear overview of a patient's history, a check that the current list of medications is consistent (this is called medication reconciliation), and a quick way to find a specific lab, imaging, or microbiology report. All three agreed on one rule: a clinician should see only the patients they are treating. They allowed a few exceptions, such as emergencies, a second opinion asked for by a colleague, and teaching. When data is reused for research or teaching, the identifying details are still removed by hand today, or through a quality-assurance or ethics process. Removing such details is called de-identification.

On trust, three things were rated absolutely essential by everyone: being able to trace each answer back to its source, having institutional or ethics approval, and complying with regulations. Keeping data inside the hospital mattered too, but less unanimously (rated 5, 4, and 2 out of 5). Everyone wanted the same kind of answer: a short summary in plain language, together with the exact source passages it is based on. Citations were seen as essential, and a response time from a few seconds up to about thirty was acceptable. The biggest worry was hallucination, meaning an answer that sounds right but is not backed by any real document. Next came unauthorized disclosure, that is, showing information to someone who should not see it. One clinician said a leak is strictly worse than a missing answer, because a missing answer mainly hurts in an emergency. Respondents also warned against the tool adding its own interpretation or moral judgment, for example about HIV status or someone's sexual orientation, and asked that it flag or refuse answers it is unsure about. Finally, they noted that institutional and legal hurdles, and plain clinical acceptance, are real obstacles to putting such a tool to use.

3.2.2 Implications for the system design

These needs translate directly into design choices, summarized together with the humanitarian needs in Table 3.1. The wish for traceable, cited, and hallucination-free answers is met by grounded retrieval, where the answer is built only from retrieved documents, by the post-generation Verifier that checks the answer against those documents, and by recording the sources in the report. The demand for plain facts without added interpretation constrains both the answer policy and the sanitizer, the component that rewrites sensitive text, to keep the facts and avoid inferring anything new. Because a leak is judged worse than an omission, the system leans towards caution: conservative defaults, and the option to stop a request it cannot make safe. The preference for data staying on-site is met by separating the privacy guarantee from where the model runs, so the same design works with a local or a remote model. Audit and compliance needs are met by an append-only report, a log that can only be added to and never edited. Restricting access to a clinician's own patients is left to the hospital's own record system, and sits outside our privacy layer.

3.3 Humanitarian expert interview

3.3.1 Findings

The interview reframed the role of privacy in humanitarian work. Confidentiality is not only about protecting individuals: it is what makes the work possible at all, earning access to affected areas and sustaining trust, dialogue, and the organization's legitimacy. The expert recalled an early field mission where teams negotiating with an armed group had to prove that they were there for humanitarian reasons, and not to gather intelligence. Sensitive information therefore goes far beyond beneficiary records: it includes context analyses, operational assessments, and politically charged material, and even data that names no one can be dangerous if it makes the organization look like it is spying. The people behind the data add a further layer of risk. They often have little real choice about the information they hand over, which can cover their religion, politics, flight history, family situation, and poverty, and its exposure can lead to arrest, persecution, or physical harm. Biometric registration, common in humanitarian operations, shows the tension well: fingerprints and iris scans speed up identification and cut fraud, but a leak of biometric data can never be undone.

These stakes are recognized in law and constrain infrastructure. The expert pointed to international case law where a humanitarian organization could not be forced to reveal mandate-related information, even before a high court. Cloud infrastructure, by contrast, was seen as a risk to trust, since under some legal regimes a provider can be ordered to hand over stored data no matter where the servers sit. Two episodes made the consequence concrete: data had to be erased at once when an African government suddenly closed an office, and on an early field mission paper records had to be burned for lack of any secure place to store them. For sensitive cases, keeping data on the organization's own infrastructure is a necessity, not a preference. Day to day, access is tightly compartmentalized (in one deployment, a press release meant formally requesting information from another department), every document carries a confidentiality label, and sensitive material moves only through approved channels. These practices follow the data-protection standards the ICRC codifies for the sector in a dedicated handbook [34]. The document searches that do happen, in the legal and policy teams that handle dialogue with armed groups and in annual donor reporting, are slow and largely manual, which makes them a promising place for privacy-preserving help.

For an AI assistant, the expert asked first for traceability: grounded retrieval with direct links to the source documents rather than freely generated text, to limit hallucination. Keeping data in the country was described as essential, and one subtle point stood out: simply asking a question already reveals what you are interested in, so the privacy of the question matters as much as the privacy of the documents. Explainable information choices, audit logs of the queries, and compliance with internal data-protection rules were all seen as important, in line with the organization's policy on the responsible use of artificial intelligence [35]. The expert was also frank about today's AI, which works unevenly across languages and in settings under-represented in its training data, and again judged a leak more damaging than a missing answer. Finally, adoption should start from a real problem rather than from the technology, as in the Engineering Humanitarian Action project that monitors armed conflicts remotely with satellite imagery [36], and the energy cost of AI was raised as a genuine concern.

3.3.2 Implications for the system design

These points map onto concrete design choices, summarized in Table 3.1. Seeing privacy as operational trust is what motivates the adversarial gate, the step that actively probes for leaks, and the auditable report. Because a leak is worse than an omission, the system again prefers caution: conservative defaults, a bounded loop that keeps trying to clean the context, and the option to abort a request it cannot clear. The need for local control is met by separating the privacy guarantee from where the model runs; a sensitive deployment can simply point the answer model at an on-premise backend, a model hosted on the organization's own machines, without changing anything else. Traceability and grounded retrieval map to the Verifier's faithfulness check and to the source fields of the report, and the wish for audit logs maps to that same report. The vulnerability profile sets the sensitive entities for the humanitarian preset: religion and politics, displacement and legal status, precise location, family and household identifiers, and economic status. The privacy of the question itself is only partly addressed: the current design cleans the retrieved documents and the answers, but not the question. We record this as a limitation and as future work.

3.4 Limitations of the requirements gathering

Both sources are indicative, not representative. The humanitarian input comes, for now, from a single expert, and reflects one organization's stance. The healthcare survey has only three respondents so far, which is a small sample. We use them to shape design choices and test scenarios, not as general empirical findings, and we plan to extend them with a review by a data-protection expert and a wider survey of clinicians.

Table 3.1: Requirements and design responses. Needs raised by the healthcare (HC) clinician survey and the humanitarian (HU) expert interview, and the mechanism that addresses each. A checkmark marks a domain that raised the need, a dash marks a need not raised by that domain.

Requirement	Raised by		Design response
	HC	HU	
Traceable, cited answers	✓	✓	Verifier faithfulness check, and sources in the report
Factual answers, no added interpretation	✓	–	Sanitization preserves facts, and verifier check after
Leak worse than omission	✓	✓	Conservative defaults, a bounded escalation loop, and abort if unsafe
Data stays under local control	✓	✓	Privacy guarantee decoupled from where the model runs
Audit and compliance	✓	✓	Per-request report
Flag or refuse uncertain answers	✓	–	Adversarial gate, user validation, verifier check
Subject vulnerability sets entities	–	✓	Humanitarian preset (religion, politics, displacement, location, ...)
Access limited to own patients	✓	–	Host record system (out of scope)
Query (question) privacy	–	✓	Partial, future work

4 Methods

This section describes the high-level design of the multi-agent privacy layer, its integration into the MMORE RAG framework, the evaluation corpora and protocol, and the ethical design constraints. Empirical results are reported in Section 5 (will be added later).

4.1 Study design

This work is a constraint-driven system-design contribution combined with a comparative methodological evaluation. The governing constraint is that no raw personal information may reach an untrusted answer model. The methodological focus is a domain-aware multi-agent privacy layer that operates at query time, between retrieval and generation, inside the MMORE RAG framework (Figure 4.1); its architecture is organized around a small set of recurring design patterns, presented in Section 4.2.

The evaluation is comparative and ablative under a fixed protocol: it compares the PII detection engines and the sanitization strategies, and ablates the two adversarial components, against two baselines, namely a no-privacy RAG pipeline and a single-pass sanitization pipeline without adversarial verification. Experiments span the global, healthcare, and humanitarian domains, with healthcare as the primary domain. The metrics, baselines, and experiment matrix are defined in the evaluation protocol (Section 4.6).

The layer was delivered incrementally, as a dependency-ordered cascade of small, independently reviewed pull requests rather than a single large change. The development workflow behind this choice is described in Section 4.5, and each pull request is documented in full in Appendix F. In dependency order, they are the following.

1. **Agent framework** (PR #285). A LangGraph-based agent runtime integrated into MMORE behind an optional dependency, with model serving cached and shared across agents.
2. **Detection toolkit** (PR #335). The four pluggable PII detection engines behind a common interface.
3. **Analyzer, Detector, and Sanitizer** (PR #337). Per-request policy selection and the detect-then-sanitize step that produces the sanitized context.
4. **Adversarial gate** (PR #338). The pre-generation Leakage Adversary, its bounded re-sanitization loop, and the human-in-the-loop approval gate.
5. **RAG integration** (PR #339). The untrusted answer-model call, the post-generation Verifier, the audit report, and the wiring of the privacy graph into MMORE's RAG pipeline.

The agents are prompt-driven small language models served through MMORE's existing model abstraction; no task-specific fine-tuning was performed, so there is no training procedure to report, and distilling the agents into smaller specialized models is left to future work.

4.2 Design patterns of the privacy layer

The privacy layer is a small multi-agent system, and its parts are easiest to understand through a handful of recurring design patterns. We introduce them in the order the pipeline uses them: first the foundation that every step shares, then how the layer decides what to protect and carries it out, then how it checks its own work before and after the answer is produced, and finally the guarantees that hold across the whole pipeline. The concrete, change-by-change implementation is documented in Appendix F.

4.2.1 The foundation: agents, tools, and a shared state

Three kinds of object make up the layer. An *agent* is a component driven by a small language model that makes a decision or a judgment; a *tool* is a plain function or model that agents call, such as a detection engine or a sanitization strategy; and a *node* is one step of the pipeline, run in a fixed order (Figure 4.1). Rather than passing values from step to step, every node reads and writes a single, typed *shared state*: the Analyzer writes the policy, the Detector writes the detected spans and a risk summary, the Sanitizer writes the cleaned context, and later steps read whatever they need. Holding all of a request's data in one place makes the flow explicit and the nodes independent of one another, which is what lets the remaining patterns stay local and simple.

4.2.2 An orchestrator and its executors

The front of the pipeline deliberately separates decision from execution. The Analyzer is an *orchestrator*: from the query and the retrieved passages it chooses a domain and writes a policy, namely which entities are sensitive, which detection engine to run, and which sanitization strategy to apply. The Detector and the Sanitizer are *executors*: they decide nothing and simply run the tools the policy names. The tools themselves are *interchangeable*, in that every detection engine and every sanitization strategy sits behind a common interface and registers under a name, so the policy selects one by name and a new tool can be added without changing any caller. Together these ideas let the policy logic and the tools evolve independently, and they make the later comparison between tools fair, since all are invoked in exactly the same way. The Adversary and the Verifier fit neither role: they are evaluation agents, which judge the sanitized context and the generated answer rather than produce them.

4.2.3 Checking the work, before and after generation

Producing a clean context is not enough, so the layer also checks it, using two *critic* agents. A critic inspects another component's output and judges it without redoing the work: before generation, the Adversary judges whether the cleaned context still leaks; after generation, the Verifier judges whether the answer leaks or drifts from the retrieved evidence. This is the *LLM-as-a-judge* idea of Section 2.3, used adversarially, so a second and independent opinion catches what the producing component might miss. When the pre-generation critic reports a leak, the layer enters a *bounded feedback loop*: it makes the policy stricter and runs detection and sanitization again, but only a fixed number of times, after which the request is declared unsafe rather than sent on. The bound guarantees that the loop always terminates, while the repetition gives it a chance to succeed. A final *human checkpoint* sits just before the answer model, where a reviewer can approve the cleaned context, reject it, or ask for a change.

A request for a change is expressed in ordinary language rather than through settings, and it is routed back to the Analyzer, which turns it into a concrete adjustment of the policy. If a clinician asks that the confidential details be replaced by plausible but fictitious values instead of being masked, the Analyzer weighs the registered sanitization tools against that intent and selects the one best suited to it, here the language-model rewrite that produces synthetic substitutes. If the reviewer instead points out a kind of information the current policy does not cover, for example an identifier specific to these documents, the Analyzer adds it to the set of sensitive entities so that it is found and removed on the next pass. In both cases the person states the goal and the Analyzer decides how to reach it, so the reviewer can steer the system without knowing which tool or setting to change.

4.2.4 Guarantees that hold across the pipeline

Three properties hold regardless of the request. The design draws a single *trust boundary* at the gate: only context that has passed the Adversary and the human review crosses it to the answer model. Because the guarantee is enforced at this line rather than by where the model runs, the model may be local or remote without weakening it. The layer is also *configuration-driven*: one file, backed by per-domain presets, fixes which entities are sensitive, which engine and strategy run, how strict the Adversary is, and which answer model is used. The same code thus serves the general, healthcare, and humanitarian domains by configuration alone. Finally, the layer is *fail-safe by default*: when a component is unsure it redacts rather than reveals, and when the loop cannot clear a request it aborts rather than proceeds, so mistakes tend toward over-redaction, not disclosure.

4.3 Placement of Privacy System

A privacy layer for RAG can act at one of two points: at indexing time, by sanitizing every document before it enters the vector database, or at query time, on the chunks returned for a specific request. This work enforces privacy at query time. Pre-indexing sanitization was rejected for three reasons.

- **It degrades retrieval.** Dense and sparse retrieval both match a raw user query against the indexed text. If documents are rewritten before indexing, e.g. with masked or substituted entities, the query and the index no longer share surface form, and relevant chunks are ranked lower or missed entirely. Sanitizing the query symmetrically does not solve this, because the transformation that hides an identifier is not the one a user would phrase, and the mismatch compounds with every entity.
- **It forces a non-adaptive policy.** At indexing time the query is unknown, so a single global sanitization must be committed before any request is seen. What counts as sensitive, however, depends on the request: the domain it falls under (clinical, humanitarian, or general), the strictness the caller asks for, and whether a field is identifying only in context. A fixed pre-indexing policy is necessarily the strictest common denominator, over-redacting for permissive requests while still unable to react to entities that become sensitive only given the query. Acting after retrieval lets the Analyzer choose the policy per request.
- **It makes corpus updates expensive.** Pre-indexing sanitization couples the privacy policy to the index: any change to the policy, the detection engines, or the sensitive-entity schema requires re-sanitizing and re-embedding the entire corpus, and every newly added document must be reprocessed before it can be served. Query-time enforcement leaves the corpus untouched, so the index is built once and the policy can change without reindexing.

The cost of this choice is stated explicitly: the vector database holds the raw corpus unchanged, so it must sit inside the data controller's trust boundary. The threat model addressed here is leakage to the untrusted answer model, not compromise of the index itself. Note that database-level protection is assumed to be provided by the deploying organization and is out of scope.

4.4 The tools and the agents

The privacy layer is a query-time multi-agent system, orchestrated as a LangGraph state graph with a typed shared state and two bounded loops (Figure 4.1). A request flows through the graph in a fixed order, and each stage reads from and writes to the shared state rather than passing values directly from one step to the next.

A *Context/Policy Analyzer* opens the pipeline. From the query and the retrieved chunks it selects a domain preset (global, healthcare, or humanitarian) and emits a per-request privacy policy: the set of sensitive entities, the strictness level, and the prompts used downstream. The comparative experiments reported later feed back into this choice, letting the Analyzer select the detection and sanitization tools best suited to a given query.

The *Detector* then runs the engines named by the policy behind a common pluggable interface, consolidating their outputs into typed PII spans and a risk assessment. Up to four engines can be enabled, ranging from rule-based detection (Presidio) through neural span extractors (GLiNER-PII and a token-classification filter) to an LLM-based detector, so that recall and precision can be traded off per domain (Table 4.1). The *Sanitizer* transforms only the detected spans, applying one of four strategies, from token masking and subject-consistent entity replacement to anonymization and synthetic rewrite, with defaults taken from the active preset (Table 4.2).

Table 4.1: Detection engines. The four PII detection engines, with an example from the synthetic note of Appendix G.

Engine	Description	Example
Presidio	Rule-based engine combining patterns, checksums, and named-entity recognition, extended here with custom clinical recognizers.	`617-555-0148` is matched as `PHONE_NUMBER` by a phone pattern.
GLiNER-PII	Neural zero-shot span extractor that matches the text against entity labels provided at inference time.	`Bobby` is extracted as `PERSON` with a confidence score.
`privacy-filter`	Transformer token classifier that tags every sub-word with a BIO privacy label.	`617-555-0148` is returned as six `private_phone` fragments to merge.
LLM detector	Small language model prompted through DSPy to return typed spans as structured output.	`metoprolol` is labeled `MEDICATION` thanks to the SLM reasoning capabilities.

Two adversarial agents bracket the answer model and carry the layer's privacy guarantee. Before generation, a *Leakage Adversary* probes the sanitized context against an attack taxonomy of direct extraction, membership

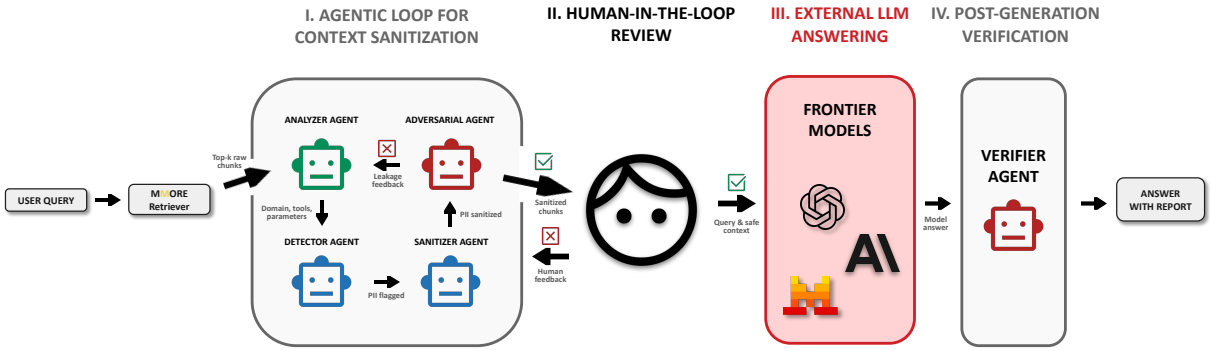


Figure 4.1: Multi-agent privacy layer integrated into MMORE. A query is answered in four stages. *(I) Agentic sanitization loop:* an Analyzer sets the privacy policy, a Detector locates sensitive spans, and a Sanitizer removes them. An Adversarial agent probes the result and, on leakage, loops back (at most N times). *(II) Human review:* a reviewer approves the cleared context or rejects it back into the loop. *(III) External answering:* the approved context is sent to the untrusted model, and the guarantee is enforced before this boundary. *(IV) Verification:* a Verifier checks the answer for residual leakage and faithfulness before releasing it with an audit report.

Table 4.2: Sanitization strategies. The four sanitization strategies, with an example from the demo of Appendix H.

Strategy	Description	Example
Token masking	Replaces each detected span with a numbered placeholder for its entity type.	`John Doe` becomes `[PERSON_1]`.
Entity replacement	Substitutes a consistent fake identity for each entity within a request.	`John Doe` becomes `David Pacheco`.
Presidio anonymization	Applies one of the Presidio operators (replace, redact, mask, or hash) to each span.	`John Doe` becomes <PERSON> or `**** Doe`.
Synthetic rewrite	A language model rewrites the passage, keeping its meaning and dropping the identifiers.	`John doe's book` becomes texttttthe man's book.

inference, and context reconstruction [2]. On detected leakage it drives a bounded escalation loop, tightening detection and sanitization for at most N iterations; if the context still cannot be cleared, the request is marked unsafe and the answer model is never called. The *answer model* itself is not an agent but a direct call to any backend through MMORE's model abstraction, local or remote; it is treated as untrusted and receives only gate-certified context, so the guarantee holds independently of where it runs. After generation, a *Verifier* inspects the raw chunks, the sanitized context, and the produced answer together, checking for residual leakage and for faithfulness to the retrieved evidence.

Every request is recorded in an append-only report that never contains raw PII. It logs the retrieved documents, the selected domain, the detection and sanitization summary, the escalation count, both adversarial verdicts, the answer backend, any human-in-the-loop events, and the final outcome, providing an auditable, regulatory-grade trail.

4.5 Development workflow, tooling, and testing

The scale of the privacy layer, delivered as a cascade of interdependent pull requests over several months, made it a natural opportunity to adopt a more disciplined development workflow than a single-author research prototype usually warrants. The engineering process was treated as part of the methodology: it is what keeps the contribution reviewable, reproducible, and safe to integrate into a shared codebase.

The pull requests listed above form a dependency cascade rather than an independent set. Each one builds on the interfaces introduced by its predecessor (the detection toolkit before the Sanitizer, the Sanitizer before the adversarial gate), so they were opened and reviewed in order against the shared `mmore/v2` integration branch. Because that branch kept moving as earlier pull requests merged, later branches were periodically rebased onto it rather than merged back and forth: rebasing replays the outstanding commits on top of the current integration state, which resolves conflicts one commit at a time and preserves a linear, readable history. A single merge into the main branch was reserved for the end, once the full cascade had been validated together.

Keeping several branches open at once is inexpensive with git worktrees, which check out each branch into its own sibling directory instead of switching a single working copy. This avoids stashing half-finished changes on every context switch. More usefully here, it also avoids rebuilding the Python virtual environment when moving between a branch under review and the branch under active development, since each worktree carries its own working tree while sharing the underlying repository history.

Correctness was guarded automatically through continuous integration. A GitHub Actions workflow (`pyright.yml`) runs static type checking on every push and pull request, so interface mismatches between the layer's components, such as a wrong return type from a `DetectionEngine` or a renamed field on `PrivacyConfig`, surface before human review rather than at run time. Static typing is particularly valuable for a system whose behavior rests on contracts between many small agents: the type checker enforces those contracts as the graph grows. A companion workflow (`tests.yml`) runs the continuous-integration-safe unit test suite described next; it too triggers on every push and pull request and executes on GitHub-hosted runners, so each contribution is exercised in a clean environment independent of any developer machine. Together the two workflows give every pull request a uniform, automatically checked quality gate before it enters the cascade.

Each pull request ships with its own unit tests. They serve two purposes: confirming that the new component behaves correctly in isolation, and acting as regression tests so that a later pull request cannot silently break an earlier one. The tests are continuous-integration-safe by default, mocking model calls, with optional gated tests that exercise real model weights when those are available. As a baseline guarantee, the privacy layer introduces no regression to MMORE when it is disabled. These per-component unit tests are a development-time scaffold; at the end of the project they will be replaced by end-to-end integration tests that exercise the full retrieve, sanitize, verify, and answer pipeline.

4.6 Evaluation

The evaluation aligns metrics with the three research questions and uses two baselines: a no-privacy RAG pipeline (with high utility but low privacy), and a single-pass sanitization pipeline without adversaries, which isolates the contribution of double adversarial verification. The experiments are summarized in Table 4.3, and their results are reported in Section 5. Every model-based tool runs locally through MMORE's own Hugging Face loader on a small language model, so the numbers reflect a modest on-premise setting, and every aggregate is reported with a 95% confidence intervals.

Detection and sanitization are evaluated on the healthcare subset of the synthetic Nemotron-PII corpus, whose gold identifier spans provide the ground truth (Appendix A). Detection quality uses precision, recall, and F1 per engine and entity type to assess performances. For sanitization, every strategy is fed the same gold spans so the strategies

are compared independently of detector quality. We then measure the residual PII leakage rate, overall and per entity type, thanks to a local cosine-similarity with the raw information.

The end-to-end effect on utility is measured on a subset of the MIRIAD dataset of medical query-response pairs [37], indexed as the retrieval corpus and supplying the clinical questions. Each question is answered with the privacy layer disabled and enabled, and the primary outcome is a utility non-regression check: mean answer quality under privacy must stay within a small margin of the no-privacy baseline. Answer quality is scored again with a cosine-similarity for answer similarity, faithfulness, and relevancy. Adversarial robustness compares leakage with and without the pre-generation gate. Latency is profiled directly for each tool.

Table 4.3: Experiments. Each experiment with the research question it addresses, the primary metrics, and the comparison baseline.

Experiment	RQ	Primary metrics	Baseline
Detection comparison	RQ2	Precision / recall / F1 per engine and entity type	–
Sanitization comparison	RQ2	PII leakage rate	Raw text
Latency profile	RQ1	Runtime percentiles (p50, p95)	–
Adversarial verification	RQ3	Latency and answer quality	Single-pass sanitization
End-to-end	RQ1	Answer quality	No-privacy RAG

Results are reported in Section 5.

5 Results

5.1 Composition of the healthcare evaluation corpus

Before benchmarking the pipeline, we ran an exploratory analysis of the Nemotron-PII dataset, focusing on its healthcare subset. The goal was to understand what the evaluation data contains. Figure 5.1 summarizes the 60 records along three axes. The subset covers many clinical document types, from admission checklists and pathology summaries to blood test reports and discharge forms. The benchmark is therefore not tied to a single kind of note. Each record is short, around 600 characters on average, and holds about seven PII entities. This high density makes the subset a good test for detection and sanitization.

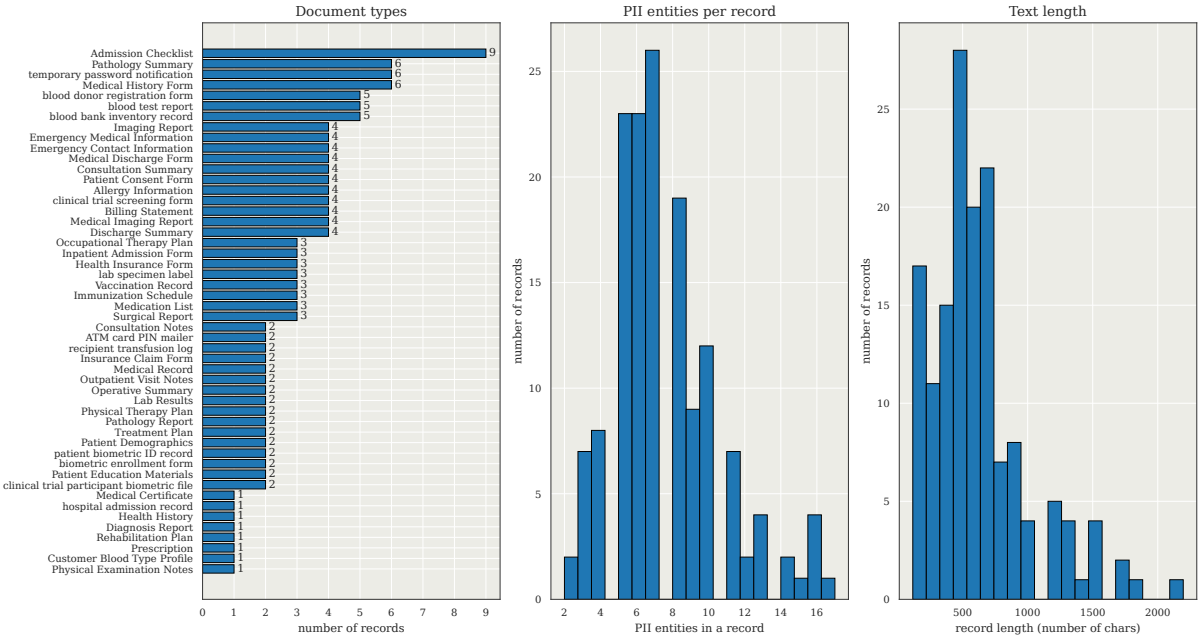


Figure 5.1: Healthcare subset at a glance. Distribution of document types (left), number of PII entities per record (middle), and record length in characters (right).

We then looked at which kinds of personal data are present. Each identifier was embedded, projected to two dimensions with principal component analysis, and grouped by K-means into nine clusters of related types. The result is shown by color in Figure 5.2. A few categories dominate, mainly names, dates, and record identifiers. Clinically specific ones also appear, such as blood type, health plan beneficiary number, and biometric identifier. The clusters match clear meaning groups, for example contact details, demographic attributes, and financial identifiers. This confirms that the subset covers the main identifier types a clinical privacy filter has to handle.

5.2 Detection engines comparison

This subsection addresses RQ2. Each engine is run over the gold-annotated healthcare corpus. Its predicted spans are matched to the ground-truth spans by character overlap, so a span counts as found when it overlaps a gold span, whatever the predicted type. The counts are pooled into canonical entity types and summarized with six metrics (Figure 5.3). Precision and recall are the base measures. F1 is their harmonic mean, micro-averaged over all spans. Macro-F1 averages the per-type F1 so that rare and frequent types count equally, and weighted-F1 weights that average by how often each type occurs. Accuracy is the ratio of correct spans to all predicted and missed spans. Every bar shows a 95% confidence interval, so overlapping intervals mark differences that may not be meaningful.

The engines fall into two groups. GLiNER, the LLM detector, and Presidio are precise but cautious. GLiNER is the most precise (around 0.98) and has the best micro-F1. The LLM detector is close behind and is the most even across types. Presidio is below both. All three find only about half of the identifiers, so recall is their main limit, not precision. The privacy filter is the opposite. It has the highest recall (around 0.71) but the lowest precision (around 0.22), because its token-level tagger splits each identifier into many sub-word spans and over-predicts. Its weighted-F1 is the best of the four, driven by the most frequent types, but its low micro- and macro-F1 show the cost in false positives. For a privacy filter a miss matters more than a false alarm, so this recall is useful, as long as the fragmented output is merged first.

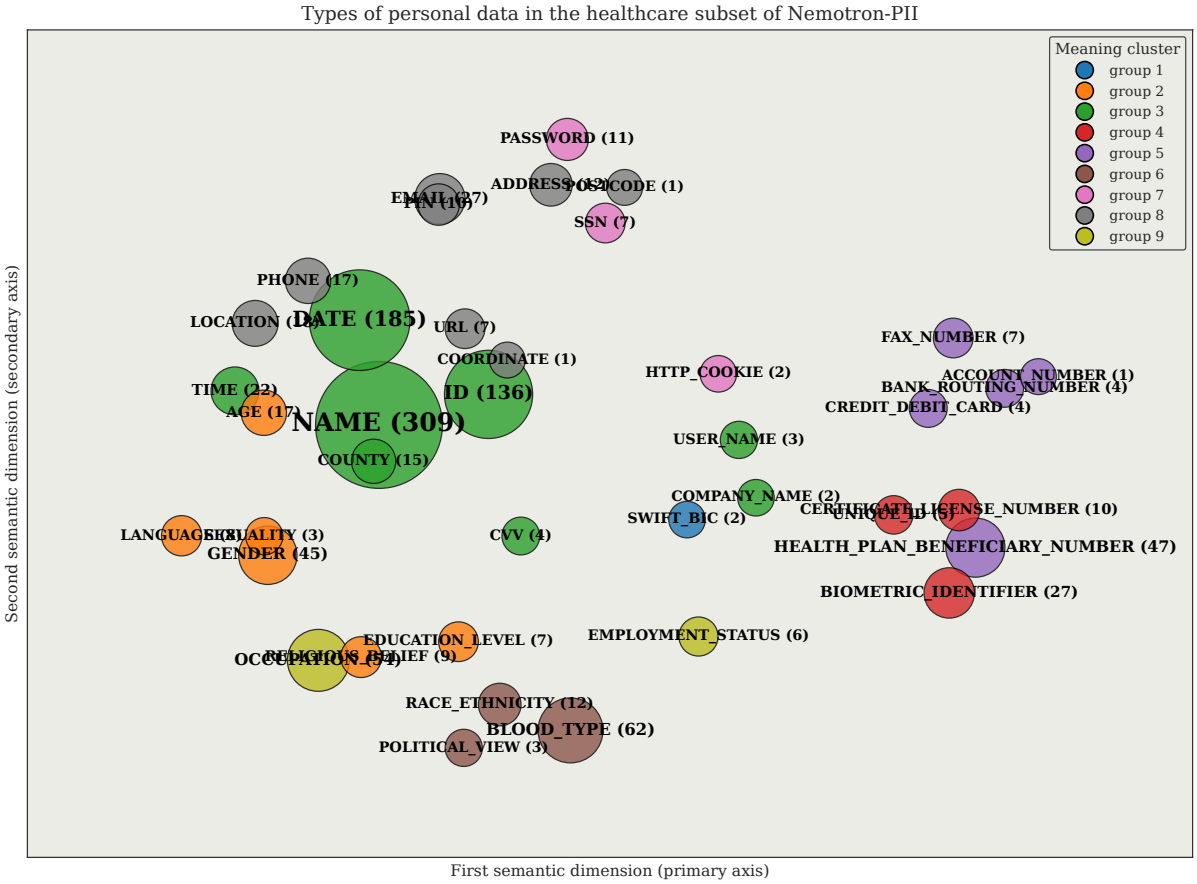


Figure 5.2: Types of personal data in the healthcare subset. Each PII type is embedded and projected with PCA, sized by frequency and colored by K-means meaning group (nine clusters).

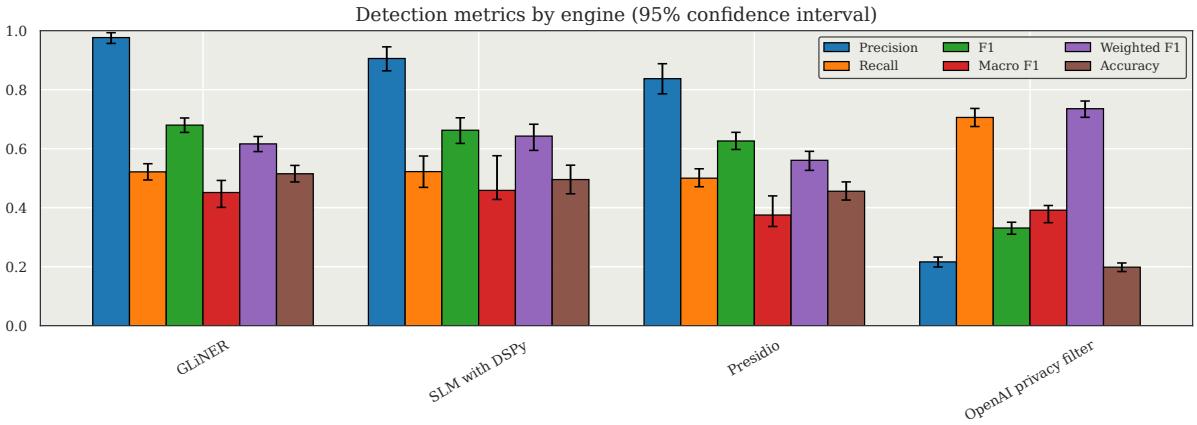


Figure 5.3: Detection metrics by engine. Precision, recall, F1 (micro), macro-F1, weighted-F1, and accuracy for each engine under overlap matching, with 95% confidence intervals.

To see where that recall comes from, Figure 5.4 breaks it down by data type. It reuses the layout of Figure 5.2, so position marks a type's meaning group, size its frequency, and color its recall. GLiNER and Presidio behave in a similar way. Both do well on common identifiers and struggle on the rarer attribute types. The LLM detector is the most even of the four and keeps decent recall even on harder categories such as health plan beneficiary and biometric identifier. The privacy filter is strong on simple, structured types like names, dates, and account numbers. It drops on attributes such as race, blood type, or political view, which are expressed in words rather than in a fixed pattern and are harder for a token-level tagger to catch.

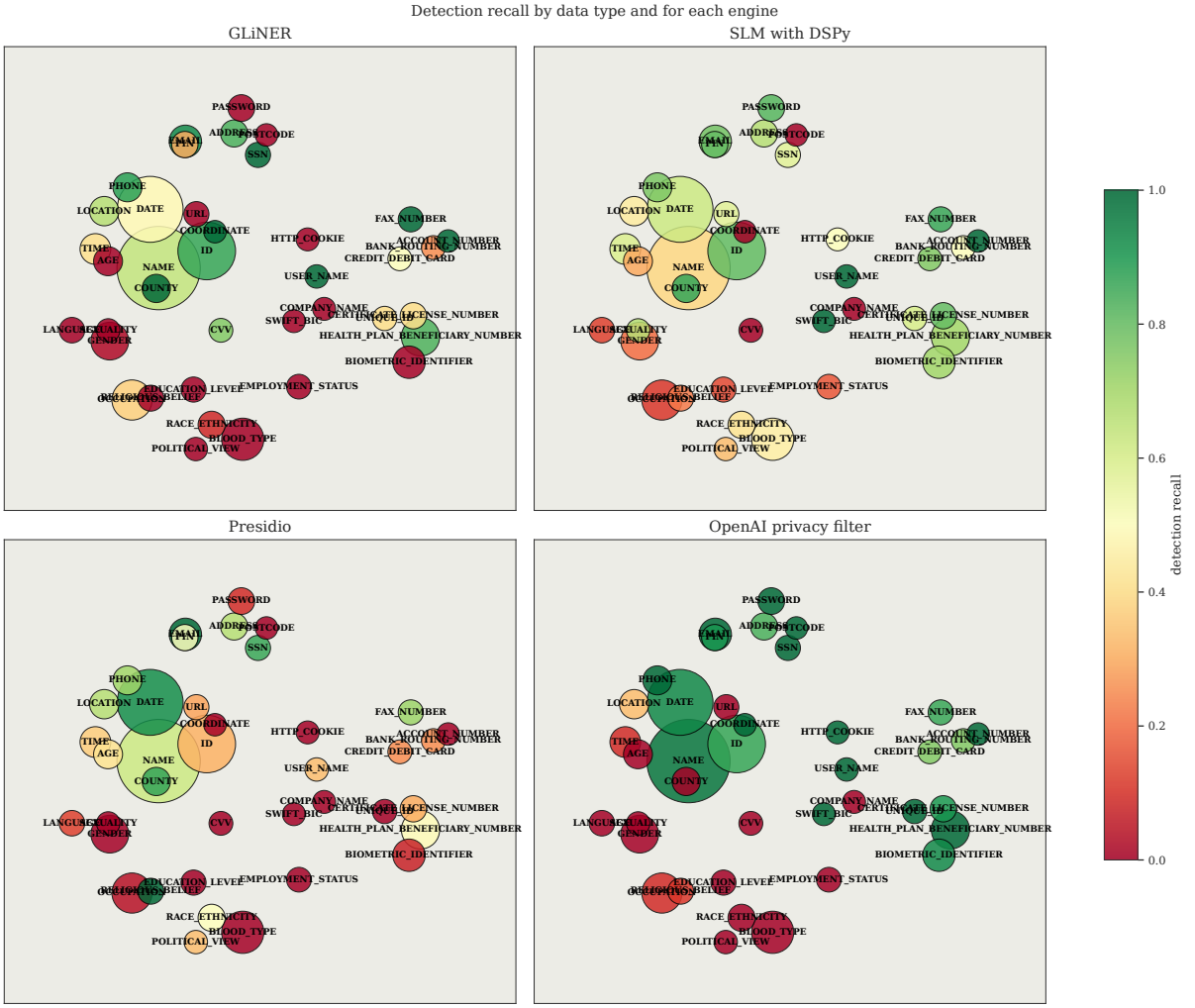


Figure 5.4: Detection recall by data type. Per-engine recall mapped onto the semantic layout of Figure 5.2. Position encodes the meaning group, marker size the frequency, and color the recall from red (0) to green (1).

5.3 Sanitization strategies comparison

This subsection addresses RQ2. Each strategy receives the text and the known PII spans, then hides them by masking, replacing, or rewriting. We measure the leakage rate, the share of the original identifiers that still appear word-for-word in the cleaned text. Lower is better. Every strategy is given the same gold spans, so the test measures the sanitizer alone and not the detector.

The three deterministic strategies, token masking, entity replacement, and Presidio, remove almost everything, with leakage close to zero (Figure 5.5). Synthetic rewrite is the clear exception at about 0.39. It paraphrases the passage with a small language model, and in doing so it sometimes restates an identifier in different words. This puts the information back into the text. Its fluency therefore comes at a real cost in privacy.

Splitting leakage by data type shows which identifiers are hard to hide (Figure 5.6). For the deterministic strategies almost every type stays safe. For synthetic rewrite the pattern is clear. Numeric and structured identifiers such as record numbers, account numbers, and phone numbers are easy to remove and stay green. Free-text personal

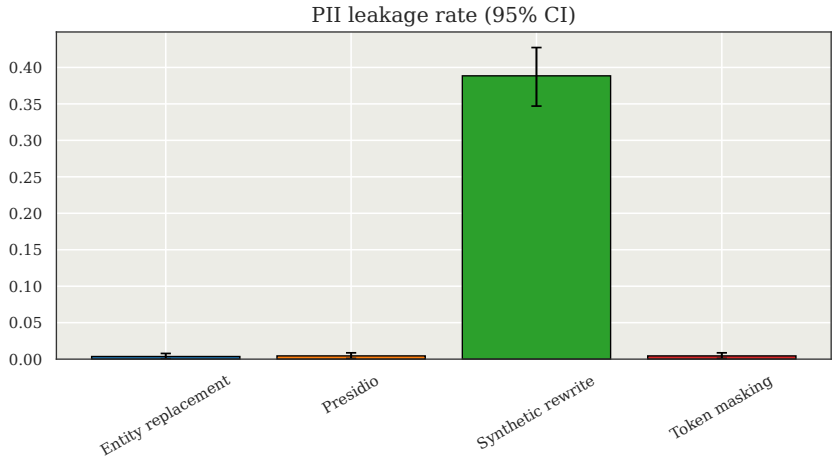


Figure 5.5: PII leakage rate by strategy. Share of gold identifiers still present verbatim in the sanitized text, with 95% confidence intervals. Lower is better.

attributes such as name, age, gender, language, and religion are the ones that leak. These are the values the model tends to put back into a fluent sentence.

5.4 The privacy pipeline preserves answer quality at a latency cost

This subsection addresses RQ1. We run the same retrieval-augmented question answering over MIRIAD medical questions twice, changing only whether privacy is active. In the no-privacy condition the model answers directly from the top-3 retrieved passages. In the privacy condition those passages first pass through the MMORE privacy pipeline before the answer is produced. This run uses Presidio detection with entity-replacement sanitization, and the leakage adversary is off. Everything else is fixed, including the local answer model (`Qwen2.5-3B`) and the retrieval depth, so any change comes from the privacy layer alone.

Answer quality is measured by three scores. Each one is a cosine similarity between sentence embeddings of the two texts. The texts are encoded with the `all-MiniLM-L6-v2` model and normalized to unit length, so a score runs from about 0 for unrelated text to 1 for near-identical meaning. Answer similarity compares the answer to the reference and reflects correctness. Faithfulness compares it to the retrieved context and reflects grounding. Relevancy compares it to the question and reflects whether it stays on topic. These are embeddings, hence they are best read as relative differences between the two conditions rather than absolute quality. We also report end-to-end latency at the median (p50) and the tail (p95), with 95% confidence intervals across questions.

Answer quality is almost unchanged (Figure 5.7). The two conditions match on all three scores, and their confidence intervals overlap. Adding the privacy layer therefore does not lower how correct, grounded, or on-topic the answers are. The cost shows up in latency instead. The privacy pipeline roughly doubles the median response time, from about 20 to 43 seconds. The p95 tail is high and varies a lot for both conditions. This overhead is expected, since the privacy layer runs a multi-agent graph of extra steps before the answer, and that adds time.

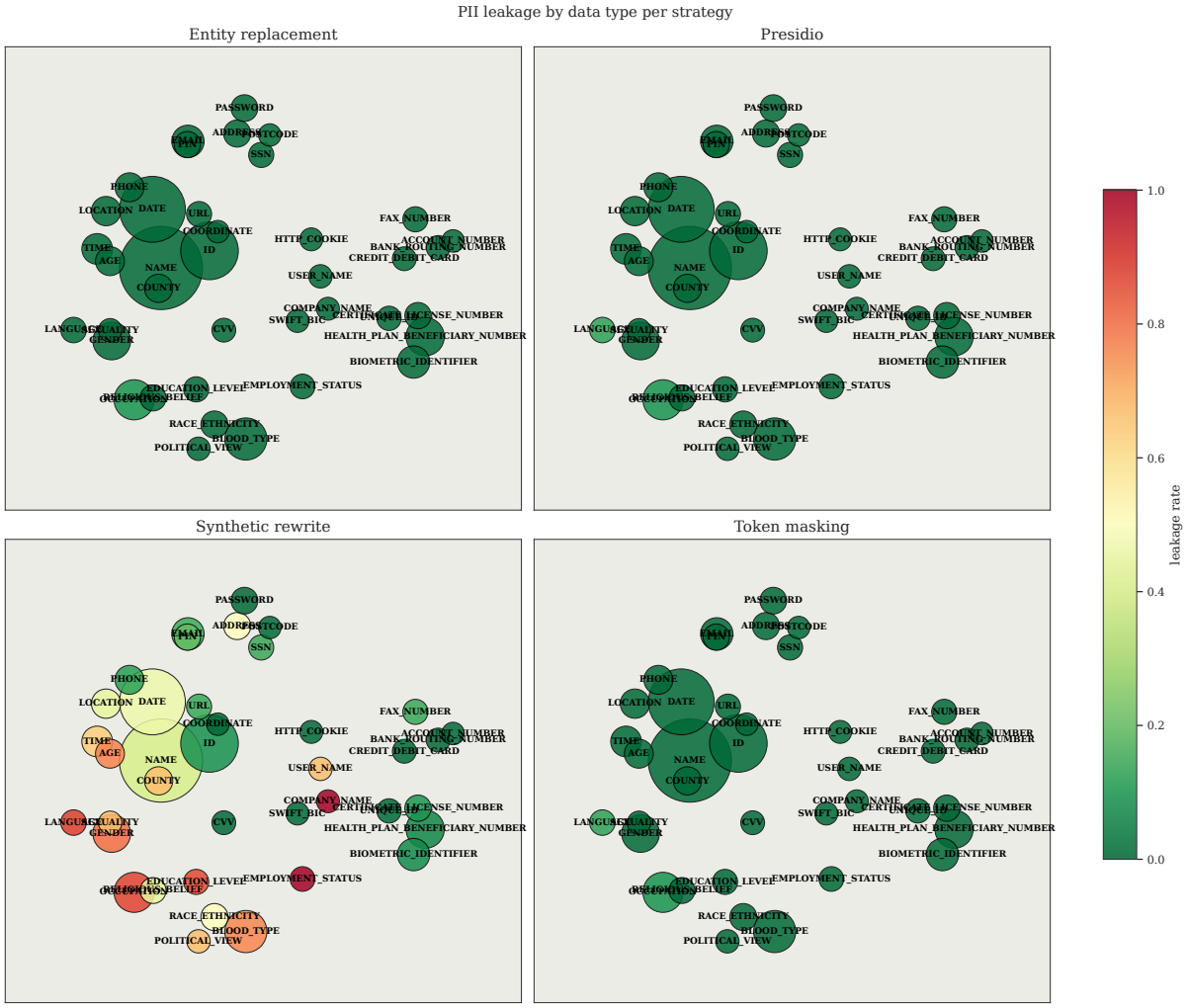


Figure 5.6: Leakage by data type. Per-strategy leakage mapped onto the semantic layout of Figure 5.2, colored from green (safe) to red (leaked), with marker size the frequency.

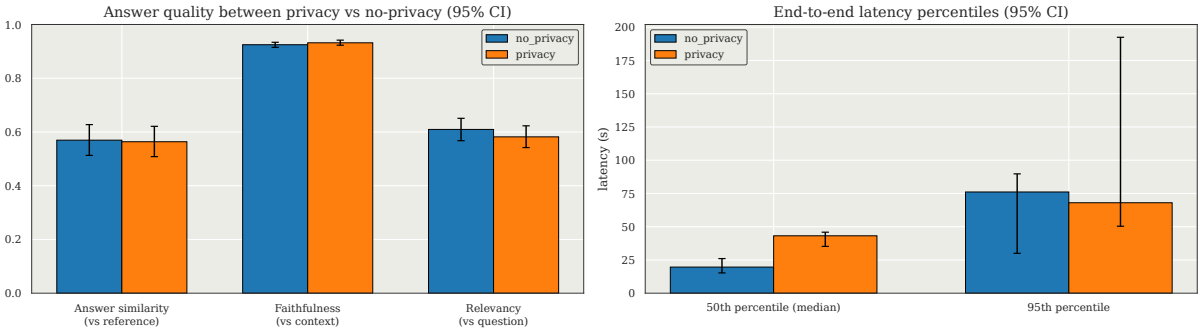


Figure 5.7: End-to-end: privacy vs no-privacy. Left: answer quality (answer similarity, faithfulness, relevancy) under each condition. Right: end-to-end latency at p50 and p95. Bars show the mean or percentile with 95% confidence intervals across questions.

6 Discussion

6.1 The adversarial agent

The two critic agents do not pay off the same way in every setup. They help most when the Detector or the Sanitizer is a small language model. The reason is how feedback moves through the pipeline. A correction can come from the human checkpoint or from the Leakage Adversary. Either way, it is written as intent and sent back to the Analyzer. The Analyzer then rewrites the policy, and with it the prompts that drive the agents. A prompt-driven agent can use that signal right away. Give it a stricter instruction, or a new example of what to hide, and it can fix its output on the next pass, with no change to code or config.

A more programmatic tool has less room to adapt. A token-classification detector or a rule-based sanitizer does not reason over feedback. It can only be re-tuned. Inside the escalation loop, that leaves two levers. We can lower the confidence threshold to catch borderline spans. We can also add more entity labels to the list of things to remove. Both help, but both hit a ceiling. Once the threshold is low enough and the label list complete, running the pass again does little. The loop can reach its limit and still miss the leak the Adversary flagged.

This gap explains where the privacy layer works best. With small language models in the detect-then-sanitize path, each adversarial verdict turns into a concrete fix. The feedback loop then tends to converge faster and fail closed less often. The adversarial design and small, steerable models go well together. The critics give the signal, and a reasoning model is what can act on it.

6.2 The catch with small models

This upside comes with a catch. A prompt-driven Detector or Sanitizer is still a generative model. Even with constrained decoding to keep the output in the right schema, the model can still break the format. When that happens, the call fails and the pipeline retries it, sometimes a few times, until the result is valid. Smaller models make this worse. The smaller the model, the more often it slips, and this matters when we pick the size of the agents. In short, small models are easy to steer but less reliable. This leaves a clear trade-off: more control over the sanitization on one side, more speed on the other.

6.3 Practical application: the CHITCHAT project

A concrete deployment target for the privacy layer is CHITCHAT, a humanitarian project that develops Humani-T, a toolkit for guiding the development and evaluation of humanitarian principles in language-model systems. Humani-T maps a proposed use case to the principles of humanity, impartiality, neutrality, and independence, surfaces ethical trade-offs and algorithmic harms across the system lifecycle, and grounds its guidance in scientific literature rather than general web sources. To make this guidance context-specific, Humani-T also lets a humanitarian organization bring its own internal material, such as policies, field manuals, protection guidance, and data-responsibility documents, into the analysis.

Humani-T uses MMORE as the document-processing backbone for that internal material: uploaded files are ingested, chunked, indexed, and retrieved through the same pipeline this work extends (Section 4). The organizational documents that make the guidance useful are precisely the ones that are confidential, since they may contain personal names, precise locations of vulnerable groups, case identifiers, protection and health details, and conflict-sensitive operational information.

This is the situation the privacy layer is designed for. The pre-generation Leakage Adversary ensures that confidential organizational content never reaches an untrusted answer model, and the post-generation Verifier checks the synthesized guidance for residual leakage and for faithfulness to the retrieved evidence. Because the privacy guarantee is decoupled from generation locality, CHITCHAT can use a strong external model while keeping raw confidential material behind the gate. The auditable report also aligns with the traceability and accountability expectations identified in the expert interview (Section 3). CHITCHAT therefore provides a realistic setting in which to validate the humanitarian domain preset and the design constraints in practice, beyond benchmark corpora. The integration of MMORE with the privacy pipeline into CHITCHAT is still ongoing currently, and the first tests will come soon.

7 Conclusion

This work set out to protect sensitive documents in retrieval-augmented generation without giving up the quality of strong external models. The resulting privacy layer is integrated into MMORE, is grounded in the requirements of clinicians and a humanitarian expert, and treats privacy as a process rather than a single pass: a policy is chosen per request, sensitive text is detected and sanitized, and adversarial checks surround the untrusted answer model. The pipeline is operational end-to-end, and its quantitative evaluation is defined.

Several questions remain open. Small local agents must catch leaks that attacks against frontier models are designed to exploit, and it is not yet clear how far this asymmetry can be pushed. The strictness of the Adversary proved sensitive to its prompt, which raises the question of how to calibrate such a judge without gold labels.

The next steps follow directly. With the practitioners' requirements established, attention should now turn to the legal ones: the criteria the system must meet to gain institutional and regulatory approval. The humanitarian preset also remains to be validated in a real deployment through the CHITCHAT collaboration. Distilling the agents into smaller specialized models would then bring the deployment cost further down and improve performances.

Acknowledgements

First and foremost, I would like to thank my supervisor, Fabrice Nemo, for his incredible dedication throughout this project. His availability, the many discussions we shared, and the clever solutions that emerged from them were a tremendous driving force in bringing this work to its current state.

I would also like to thank my colleagues at the LiGHT lab, with whom work sessions sometimes turned into moments of relaxation. I keep fond memories of our board game session over a traditional Swiss raclette, and of the evening we all spent at an Ethiopian restaurant to celebrate the end of the semester.

I am also grateful to my friends, who lifted my spirits and helped me take my mind off things in the moments when the path ahead seemed a little less clear.

Finally, I thank my family, who has been unfailingly supportive and loving throughout, and who allowed me to keep my motivation from beginning to end. Thank you all.

References

[1] Yunfan Gao et al., *Retrieval-augmented generation for large language models: A survey*, 2024. arXiv: 2312.10997 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2312.10997

[2] Shaowei Guan, Hin Chi Kwok, Ngai Fong Law, Gregor Stiglic, Harry Qin, and Vivian Hui, *Privacy Challenges and Solutions in Retrieval-Augmented Generation-Enhanced LLMs for Healthcare Chatbots: A Review of Applications, Risks, and Future Directions*, arXiv:2511.11347, Nov. 2025. [Online]. Available: https://arxiv.org/abs/2511.11347

[3] U.S. Department of Health and Human Services, Office for Civil Rights, *Guidance regarding methods for de-identification of protected health information in accordance with the health insurance portability and accountability act (HIPAA) privacy rule*, 45 C.F.R. § 164.514(b)(2), 2012. [Online]. Available: https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/index.html

[4] European Parliament and Council of the European Union, *Regulation (EU) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*, OJ L 119, 4.5.2016, pp. 1–88, 2016. [Online]. Available: http://data.europa.eu/eli/reg/2016/679/oj

[5] European Parliament and Council of the European Union, *Regulation (EU) 2024/1689 of the european parliament and of the council of 13 june 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*, OJ L, 12.7.2024, 2024. [Online]. Available: http://data.europa.eu/eli/reg/2024/1689/oj

[6] Pengcheng Zhou, Yinglun Feng, and Zhongliang Yang, *Privacy-Aware RAG: Secure and Isolated Knowledge Retrieval*, arXiv:2503.15548, Mar. 2025. [Online]. Available: https://arxiv.org/abs/2503.15548

[7] Yujing Wang, Hainan Zhang, et al., *Learning to Erase Private Knowledge from Multi-Documents for Retrieval-Augmented Large Language Models*, arXiv:2504.09910, Apr. 2025. [Online]. Available: https://arxiv.org/abs/2504.09910

[8] Yihang Cheng, Lan Zhang, Junyang Wang, Mu Yuan, and Yunhao Yao, "RemoteRAG: A Privacy-Preserving LLM Cloud RAG Service," in *Findings of the Association for Computational Linguistics (ACL)*, 2025. [Online]. Available: https://aclanthology.org/2025.findings-acl.197/

[9] Haoran Wang, Xiongxiao Xu, Baixiang Huang, and Kai Shu, *Privacy-Aware Decoding: Mitigating Privacy Leakage of Large Language Models in Retrieval-Augmented Generation*, arXiv:2508.03098, Aug. 2025. [Online]. Available: https://arxiv.org/abs/2508.03098

[10] Tatsuki Koga, Ruihan Wu, Zhiyuan Zhang, and Kamalika Chaudhuri, *Privacy-Preserving Retrieval Augmented Generation with Differential Privacy*, arXiv:2412.04697, Dec. 2024. [Online]. Available: https://arxiv.org/abs/2412.04697

[11] Shenglai Zeng et al., "Mitigating the Privacy Issues in Retrieval-Augmented Generation (RAG) via Pure Synthetic Data," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. [Online]. Available: https://aclanthology.org/2025.emnlp-main.1247/

[12] Thejan Bandara Weerasekara, Chinthani Chandeepa, Oshan Sandeep Amarasuriya, and Chathuranga Hettiarachchi, "EdgeCare: Privacy-Preserving Medical Advising System on Mobile Devices," in *IEEE/ACM International Conference on Connected Health: Applications, Systems and Engineering Technologies (CHASE)*, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11217132

[13] Patrick Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. arXiv: 2005.11401 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2005.11401

[14] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang, "REALM: Retrieval-augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. arXiv: 2002.08909 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2002.08909

[15] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.11511 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2310.11511

[16] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang, "Benchmarking retrieval-augmented generation for medicine," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. arXiv: 2402.13178 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2402.13178

[17] Akihiko Wada et al., "Retrieval-Augmented Generation Elevates Local LLM Quality in Radiology Contrast Media Consultation," *npj Digital Medicine*, 2025. DOI: 10.1038/s41746-025-01802-z [Online]. Available: https://www.nature.com/articles/s41746-025-01802-z

[18] Alexandre Sallinen et al., *MMORE: Massive multimodal open RAG & extraction*, 2025. arXiv: 2509.11937 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2509.11937

[19] Nicholas Carlini et al., "Extracting training data from large language models," in *30th USENIX Security Symposium (USENIX Security 21)*, USENIX Association, 2021, pp. 2633–2650. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting

[20] Siwon Kim, Sangdoo Yun, Hwaran Lee, Martin Gubri, Sungroh Yoon, and Seong Joon Oh, "ProPILE: Probing privacy leakage in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv: 2307.01881 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2307.01881

[21] Shenglai Zeng et al., "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. arXiv: 2402.16893 [cs.CL]. [Online]. Available: https://aclanthology.org/2024.findings-acl.267/

[22] Atousa Arzanipour, Rouzbeh Behnia, Reza Ebrahimi, and Kaushik Dutta, *RAG Security and Privacy: Formalizing the Threat Model and Attack Surface*, arXiv:2509.20324, Sep. 2025. [Online]. Available: https://arxiv.org/abs/2509.20324

[23] Yiqun Wei, Pengcheng Xia, Yiyang Ni, and Jun Li, "PE-RAG: Privacy-Preserving LLM-based RAG with Split Inference and Masked Privacy Recovery," in *IEEE/CIC International Conference on Communications in China (ICCC)*, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11148675

[24] Cheng Qian, Hainan Zhang, Yongxin Tong, Hong-Wei Zheng, and Zhiming Zheng, *HyFedRAG: A Federated Retrieval-Augmented Generation Framework for Heterogeneous and Privacy-Sensitive Data*, arXiv:2509.06444, 2025. [Online]. Available: https://arxiv.org/abs/2509.06444

[25] Xi Fang, Liang Qiao, Jun Shi, and Hong An, "Guardian Angel: A Secure and Efficient Retrieval-Augmented Generation Framework," in *International Conference on Artificial Intelligence and Information Technology Applications (AIITA)*, 2025. DOI: 10.1109/AIITA65135.2025.11047845 [Online]. Available: https://ieeexplore.ieee.org/document/11047845

[26] Mark van Velzen et al., "Privacy-, Linguistic-, and Information-Preserving Synthesis of Clinical Documentation through Generative Agents," *Frontiers in Artificial Intelligence*, 2025. [Online]. Available: https://www.frontiersin.org/articles/10.3389/frai.2025.1644084/

[27] Shunyu Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. arXiv: 2210.03629 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2210.03629

[28] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv: 2303.11366 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2303.11366

[29] Qingyun Wu et al., *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*, 2023. arXiv: 2308.08155 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2308.08155

[30] Taicheng Guo et al., "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 8048–8057. arXiv: 2402.01680 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2402.01680

[31] Omar Khattab et al., "DSPy: Compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.03714 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2310.03714

[32] Lianmin Zheng et al., "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv: 2306.05685 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2306.05685

[33] Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaei Khoei, and Athanasios V. Vasilakos, *Agentic retrieval-augmented generation: A survey on agentic RAG*, 2025. arXiv: 2501.09136 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2501.09136

[34] International Committee of the Red Cross, *Handbook on data protection in humanitarian action*, 2nd edition, eds. Christopher Kuner and Massimo Marelli, 2020. [Online]. Available: https://www.icrc.org/en/data-protection-humanitarian-action-handbook

[35] International Committee of the Red Cross, *Building a responsible humanitarian approach: The ICRC's policy on artificial intelligence*, ICRC Policy Publication, 2024. [Online]. Available: https://www.icrc.org/en/publication/building-responsible-humanitarian-approach-icrcs-policy-artificial-intelligence

[36] Engineering Humanitarian Action, *Remote monitoring of armed conflicts*, Case study. [Online]. Available: https://eha.swiss/case-study/remote-monitoring-of-armed-conflicts/

[37] Qinyue Zheng et al., *Miriad: Augmenting llms with millions of medical query-response pairs*, 2025. arXiv: 2506.06091 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2506.06091

A Code and data availability

Code. All of the work described in this thesis is public and lives in the main branch of the MMORE repository at `https://github.com/EPFLiGHT/mmore`. The privacy layer was built as a series of pull requests tracked under a main issue, issue #287, which links each sub-issue to the pull request that closes it and gives a self-contained view of how the layer was assembled.

Detection and sanitization data. The pipeline's ability to detect and sanitize personally identifiable information was evaluated on the Nemotron-PII dataset (`https://huggingface.co/datasets/nvidia/Nemotron-PII`). It is a large synthetic corpus of documents seeded with a wide range of PII types and released with span-level annotations of every identifier, which makes it a convenient ground truth for measuring detection and redaction quality without touching real patient records. Since this thesis targets the clinical setting, we used only the healthcare-related samples of the dataset.

An exploratory analysis of this subset, covering the document types, the frequency of each identifier type, and the density of identifiers per document, is reported in the Results (Section 5.1) to characterize the evaluation material.

Retrieval data. The effect of the privacy layer on retrieval-augmented generation, measured with and without the privacy pipeline activated, was evaluated on MIRIAD (`https://huggingface.co/datasets/miriad/miriad-5.8M`), a curated corpus of about 5.8 million medical question-answer pairs distilled from peer-reviewed literature and designed for retrieval and grounding in the medical domain. Running against the full corpus was unnecessary for our purpose, so we drew a subset of question-answer pairs and used the associated passages as the retrieval corpus for the end-to-end comparison.

B Healthcare clinician needs survey

Privacy-preserving AI system for medical documents

Healthcare professional needs survey

Question 1: Current practice

1a. In your daily work, what types of questions or searches do you most often need to run in patient records (e.g. past diagnoses, laboratory trends, medication history, referral letters)?

1b. What tools or methods do you currently use to run these searches?

1c. What are the main limitations or sources of frustration you encounter with these tools, if any?

Question 2: Information access and sharing

2a. In your institution, should a physician be able to access detailed information only for the patients they directly care for?

☐ Yes, physicians should only have access to their direct patients.

☐ Yes, but with exceptions (e.g. emergencies, specialist opinions requested by a colleague).

☐ No, physicians can generally access the records of any patient within the department or institution.

☐ Other: _____

2b. In your current practice, how do you share patient information with colleagues when necessary? (multiple answers possible)

☐ Direct access via centralised software.

☐ Printed documents.

☐ Oral transmission (phone, in person).

☐ Secure messaging (internal chat, secure e-mail).

☐ Other channels (regular e-mail, SMS, WhatsApp, etc.).

☐ Other: _____

2c. When patient information is reused for secondary purposes (e.g. research or teaching), how is sensitive data currently removed or masked before it is shared (e.g. manually by the clinician, by a dedicated person, using a specific tool, through a dedicated process)?

Question 3: Practical case

Imagine that the AI tool gives you an overview of the clinical history of a specific patient. In answering the following, you may draw on your experience with AI-based conversational agents: what worked well, what fell short, and what an ideal version of such a tool would be in your view.

3a. What would make this summary genuinely useful for you (e.g. structure, choosing the level of detail, asking follow-up questions or drilling down, refining the scope, direct links to source documents)?

3b. Conversely, what would make such an overview insufficient or unusable for you (e.g. missing or outdated information, hallucinations or invented details, no way to trace back to sources, results that are too generic or too verbose, a rigid structure that does not adapt to your question)?

Question 4: Trust in the tool

4a. Rate the importance of each factor for your trust in this tool, from 1 (*desirable*), 2 (*somewhat important*), 3 (*important*), 4 (*very important*), to 5 (*absolutely essential*).

Factor	**1**	**2**	**3**	**4**	**5**
Data stays within the hospital infrastructure	☐	☐	☐	☐	☐
Every answer is traceable to the exact source documents	☐	☐	☐	☐	☐
Audit logs of all queries	☐	☐	☐	☐	☐
The tool explains how the answer was produced	☐	☐	☐	☐	☐
Validation by the institution or ethics committee	☐	☐	☐	☐	☐
Compliance with data-protection regulations	☐	☐	☐	☐	☐

4b. Are there other elements you can think of that would be necessary for you to trust such a tool with access to patient data (technical, organizational, legal, or ethical)?

Question 5: Daily workflow

5a. When you ask the AI tool a question, what is the maximum response time you would still consider acceptable?

☐ Less than 2 seconds.

☐ 2–5 seconds.

☐ 5–15 seconds.

☐ 15–30 seconds.

☐ More than 30 seconds.

5b. How do you prefer the answer to be presented?

☐ A short natural-language summary.

☐ A longer natural-language answer.

☐ A list of relevant excerpts from the source documents.

☐ Other: _____

5c. How important is it that each answer include citations to the source documents?

☐ Does not matter.

☐ Somewhat important.

☐ Very important.

☐ Essential.

Question 6: Errors and uncertainty

The tool would be used by different types of users (physicians, researchers, administrators, etc.), each with different access rights to patient data. Errors can therefore take two forms.

6a. Which type of error would be most problematic in your work?

☐ The tool exposes information about a patient to a user who should not have access to it.

☐ The tool omits or hides information from a user who is legitimately entitled to it.

☐ Both are equally problematic.

6b. Please explain your answer. In which situations is one risk more critical than the other?

6c. When the tool is uncertain of its answer, how should it express this? (select the option(s) most relevant to you)

☐ Display a confidence score.

☐ Highlight the specific parts of the answer that are uncertain.

☐ Offer possible alternative answers when confidence is low.

☐ Refuse to answer rather than give an uncertain response.

☐ Use a visual indicator or warning icon.

☐ Add an explicit natural-language statement (e.g. “I am not sure about. . . ”).

☐ Other: _____

6d. What would help you most to verify an answer before relying on it? (select the option(s) most relevant to you)

☐ Inline citations linking each statement to the source document.

☐ Direct access to the full source documents used.

☐ A summary of the documents that were searched and those that were selected.

☐ An explanation of how the answer was derived from the sources.

☐ A log showing each step the tool followed to reach the answer.

☐ Other: _____

Question 7: Limitations and final remarks

7a. Are there situations in which you would not be allowed (legally, institutionally, or ethically) to use such a tool? Which ones?

7b. Is there anything else you would like us to take into account? Remarks, concerns, related experiences, references to existing tools or studies, or aspects of AI in healthcare you feel have been omitted. Feel free to write as much or as little as you like.

C Software engineering contributions to MMORE

Beyond the privacy layer that forms the main research contribution, this thesis also included several independent engineering tasks on the MMORE codebase. These were not research experiments, but they were a substantial part of the work, and each followed the same method: identify the problem, weigh the design options, implement the change, and validate it. Each entry below states its purpose, the approach taken, and the outcome, with a link to the change on GitHub. Pull request and issue numbers refer to the public MMORE repository.

PR 1 – Dependency compatibility failure `[GitHub]`

Purpose. Restore the MMORE processing and indexing pipelines, which had stopped running after an automated dependency update (PR #240). The update bot upgrades packages on a schedule without checking that the new versions remain compatible with one another.

Approach. The incompatible packages were pinned back to a working combination. The failure also revealed a testing gap: the pipelines were exercised only through mocked components, so a broken set of dependencies could pass the automated checks while breaking every real run.

Outcome. The pipelines run again. The lesson is that automated dependency updates need a manual compatibility check, and that tests closer to a real run would have caught the problem earlier.

PR 2 – Docker image build improvements `[GitHub]`

Purpose. Make the container images faster to build and usable on more platforms (PR #243). Builds were limited to a single Linux distribution and rebuilt everything even when only the source code had changed, which is slow on the target computing infrastructure.

Approach. The build was reorganized so that the costly step of installing dependencies is cached and reused as long as the dependencies themselves do not change; only the much cheaper installation of the project source is repeated when the code changes. Support for two further Linux distributions was added, and the images are now built for both common processor architectures and published to a public registry.

Outcome. Rebuilds after a code-only change are about ten times faster, and the images run on a wider range of systems, which helps reproducible deployment across the heterogeneous machines used in this work.

PR 3 – Table-aware chunking `[GitHub]`

Purpose. Keep tables intact when documents are split into retrieval chunks (PR #252). Tables were previously cut as if they were plain text, which separated a table's header from its rows and lowered retrieval quality on structured clinical content.

Approach. The chunker now detects tables and handles them separately from the surrounding text. The guiding idea is that a row is only meaningful together with its header, so every table chunk keeps the header attached. A configuration option chooses how finely tables are split, from one row per chunk (the default) to grouping several rows or keeping a small table whole. In the default mode, each row therefore becomes its own chunk carrying the header:

```
header + row 1 -> chunk 1   header + row 2 -> chunk 2   header + row 3 -> chunk 3
```

Outcome. Structured content stays coherent through chunking, so the header and row context that clinicians rely on is preserved in the retrieved evidence.

PR 4 – Web search pipeline optimization [GitHub]

Purpose. Remove wasted work from the optional web-search component (PR #259). It used to gather all candidate text snippets and then trim them by length, which cut text mid-sentence and spent model calls summarizing material that was never used.

Approach. Snippet collection is now bounded by how much text the downstream model can actually use: snippets are added until that budget is reached and then collection stops, instead of trimming after the fact. Duplicate results are discarded, and each search round refines the previous answer rather than starting over.

Outcome. Web search is faster for both supported providers with no loss in answer quality, and the answers are more relevant.

PR 5 – Incremental reprocessing [GitHub]

Purpose. Avoid reprocessing a whole corpus when only a few files have changed (PR #268, issue #244), which is wasteful on the large document archives used here.

Approach. A run can be pointed at the results of a previous run. A file is reprocessed only if it is new or has been modified since it was last processed; unchanged files are reused and deleted files are dropped. A related bug that duplicated entries across repeated runs was fixed at the same time.

Outcome. Only new or modified files are processed, which gives a large speedup on repeated runs over large datasets and shortens the evaluation loop used in this work.

PR 6 – Asynchronous multi-GPU processing and indexing [GitHub]

Purpose. Make the live-retrieval API usable under concurrent load and let it use all of the GPUs a machine has (PR #328, issue #324, issue #326). In production the API processed each upload synchronously, so a single user uploading a document blocked every other request until that document had been fully processed and indexed. This is acceptable for a single-user demo but not for a shared service, because a document can take minutes to process and the whole API is unresponsive during that time. There is a second waste: machines in the target environment often carry several GPUs, yet the pipeline used only one at a time, so most of the hardware sat idle while a single document was handled.

Approach. The central idea is to separate accepting a request from doing the work. An upload no longer runs the pipeline inline; it registers a job and returns immediately with a job identifier, and the processing and indexing then happen in the background. A client learns the result in one of two ways: by polling a status endpoint, or by subscribing to a stream that pushes status changes as they occur. Rejections are made explicit rather than left to a timeout: a second upload of a file that is already in flight is refused, and a saturated queue asks the client to retry later.

A small in-memory job queue coordinates the work. It keeps one worker per GPU, and each job reserves a device for its whole run, so the models loaded on a given GPU are never used by two jobs at once. Jobs beyond the available workers wait in the queue, and finished jobs stay queryable for a short while before being dropped. The queue state lives only in memory, so the logs are kept as the durable record of what ran. A tunable setting allows more than one job per GPU, which overlaps the CPU-bound and GPU-bound phases for higher utilization, at the cost of loading a second copy of the models. The request lifecycle is:

```
upload -> job id (accepted) -> queue -> per-GPU worker -> processed + indexed
                status read by polling or from a pushed event stream
```

Running the pipeline on several GPUs at once required loading one copy of each model per device, for both the document processors and the two embedding models, so that independent jobs on different GPUs neither share nor contend for the same weights. Concurrent writes to the index became a requirement as well: the lightweight single-process vector store used for local runs cannot accept writes from several workers at the same time, so the production configuration was switched to a standalone database server that can. A pre-existing bug that processed API uploads twice was fixed along the way.

Outcome. One user's upload no longer stalls the service: the request returns at once and the work proceeds in the background. Documents are processed and indexed in parallel across all visible GPUs instead of one at a time, and

progress is observable both in the logs and through the status endpoints. In a demonstration on a four-GPU server with three GPUs made available to the indexer, a single bulk upload of four large PDFs (about 450 pages each) was dispatched across the three GPUs concurrently, and the fourth document waited in the queue until a GPU became free.

D Supervision of external contributions: the Cassiopée projects

Cassiopée is the second-year project program of the Télécom SudParis engineering curriculum. Over five months, around 200 students work in teams of two or three on about a hundred projects proposed by faculty members or partner companies, covering research, development, and industrial topics. One of these projects was dedicated to MMORE: a student team contributed new features and benchmarks to the framework over the course of a semester.

Together with Fabrice Némo, we held weekly meetings with the students, Jérémy Doumeng, Mathieu Bonnet, Arthur Perrin, and Chadi Ait Ekioui, to synchronize their work with the ongoing development of MMORE. Beyond these meetings, we stayed in regular contact with the team so that their contributions would integrate smoothly into the codebase. When their project came to an end, we reviewed all of their pull requests (Table D.1) and tested the features both locally and on the EPFL RCP and CSCS clusters.

Collaborating with these students was rewarding in both directions: their different backgrounds brought fresh perspectives to the project, and advising them on productivity and code quality was a valuable supervision experience.

Table D.1: Cassiopée pull requests reviewed. Each pull request with a short description and its author.

PR	Description	Author
PR #283	Qdrant vector-store backend	J. Doumeng
PR #281	Images extracted from documents as inputs to the final LLM answer	C. Ait Ekioui
PR #308	Nemotron VLM processor as an alternative PDF backend	A. Perrin
PR #305	ColVision retrieval models (ColQwen2, ColQwen2.5, ColGemma3, ColSmol)	M. Bonnet
PR #316	Judge type-checking fixes and Hugging Face JSON parsing	C. Ait Ekioui
PR #312	LLM-as-a-judge corrective retrieval in the RAG pipeline	C. Ait Ekioui
PR #235	DuckDuckGo search replacement and optional Tavily provider	H. Satyavardhan
PR #298	Interactive terminal UI for MMORE commands	A. Perrin
PR #272	CPU/GPU test split and real Milvus integration tests	A. Perrin

E Production support: the MOOVE deployment

MOOVE, for Massive Open Online Validation and Evaluation (jointhemoove.org), is a global, expert-led platform developed by the LiGHT lab in which clinicians and humanitarian experts evaluate medical language models against their own practice. It supports the Meditron effort: doctors from around the world, especially in low-resource settings, probe the models with challenging questions and critically evaluate the answers, which keeps model development grounded in real-world clinical practice.

MOOVE's RAG system was recently migrated to MMORE, which now runs in production on the EPFL RCP clusters. As is common after a migration, problems that had not been anticipated surfaced once real users exercised the new stack. The work presented in this thesis made the MMORE codebase familiar ground, so we were able to help resolve these issues as they were raised. One early problem was that uploaded files were not persisted, so documents disappeared whenever the service restarted. The Docker images also caused trouble, with compatibility problems between the published images and the production environment. More recently, it turned out that processing, post-processing, and indexing used only a single GPU even on machines that have several. This last issue was resolved by the asynchronous multi-GPU processing and indexing change described in Appendix C (PR #328): in short, an upload no longer blocks the whole service while it is processed, and the work is now spread across all the GPUs a machine has instead of leaving most of them idle.

F Privacy-layer implementation

This appendix presents the privacy layer one pull request at a time, in the order in which they were built, matching the overview in the study design. Each entry gives the purpose of the change, the design decisions behind it, and

how it connects to the rest of the pipeline. The emphasis is on intent and structure rather than implementation detail, since the code itself is public.

PR 1 – Agent framework integration [GitHub]

Purpose. Put in place the orchestration layer that the whole privacy pipeline runs on (PR #285, issue #291). Every later component, the Analyzer, Detector, Sanitizer, and the two adversaries, is built on this shared foundation.

Approach. The agent runtime, built on the LangGraph orchestration framework, is added as an optional feature, so a standard MMORE installation is unaffected and only privacy deployments pull in the extra dependencies. Agents are described by configuration rather than by hand-written wiring: an agent is simply a name, a chosen model, and a set of tools it may call. Two engineering concerns are handled once, centrally, so that every later component inherits them. First, an agent can pause and later resume from a saved state, which is what makes the human review step possible. Second, models are loaded only when first needed and shared across agents through a common cache, so the same model is never loaded twice and hardware use stays low.

PR 2 – Detection tools implementation [GitHub]

Purpose. Provide the detection toolkit, the set of PII detectors that the Detector agent later calls as tools (PR #335, issue #292, building on PR #285). four interchangeable engines are offered, chosen to span the range from precise rule-based matching to higher-recall learned models: GLiNER, a token-classification privacy filter, Microsoft Presidio with custom clinical rules, and a general language model prompted for structured output.

Approach. All engines share a single interface: each takes the same configuration and returns the same kind of typed span list, and each registers under a name so an agent can select it without any change to the calling code. Adding a new detector therefore means implementing the interface and registering it, and nothing else. Model weights are handled by a shared cache that loads a model once, reuses it across engines and agents, and releases the least-recently-used model when memory runs short.

Illustration. On a synthetic clinical note that packs many identifier types into short, abbreviated prose, Table F.1 shows how the four engines label a representative subset of its identifiers. The full note and the complete per-engine outputs are reproduced in Appendix G.

Table F.1: Detection example. Label assigned by each engine to a representative subset of the identifiers in Listing 3. A dash marks an identifier the engine did not detect. In total the engines returned 15, 78, 26, and 21 spans; the token-classification filter tags individual sub-words, which inflates its count.

Text	GLiNER	privacy-filter	Presidio	LLM (DSPy)
`Bobby`	PERSON	private_person	PERSON	PERSON
`617-555-0148`	PHONE	private_phone	PHONE_NUMBER	PHONE
`Linwood Ave`	–	private_address	–	–
`Austin`	LOCATION	–	LOCATION	–
`metoprolol`	–	–	–	MEDICATION
`St. Mary's`	LOCATION	–	PERSON	LOCATION
`04/23/1955`	DATE	private_date	DATE_TIME HOSPITAL_DATE	–
`12345678`	MRN	account_number	MRN	MRN
`1245-6788`	–	–	DATE_TIME	MRN
`BCXY 99-88-77`	INSURANCE_ID	account_number	–	INSURANCE_ID
`jsmith@hosp-derm.org`	EMAIL	private_email	EMAIL_ADDRESS URL	EMAIL
`xxx-xx-4321`	SSN	account_number	–	SSN
`c. Tentative d/`	–	–	PERSON	–

No engine catches everything, and the disagreements are informative. GLiNER returns clean entity-level spans. The token-classification filter tags individual sub-words, so a single phone number is split into many fragments that must be merged. Presidio adds clinical categories such as medical record numbers and insurance identifiers, but it also produces occasional false positives and here missed the social security number. The language-model engine adds clinically meaningful categories such as medication and recovered a mistyped record number, but missed both dates of birth. This spread is what motivates offering several engines and, in a later change, combining their outputs rather than trusting any single detector.

PR 3 – Add privacy agents: Analyzer, Detector, Sanitizer [GitHub]

Purpose. Produce the first sanitized context: choose the domain, decide what is sensitive, find it, and transform it (PR #337, issue #293, building on PR #335). This adds the first three stages of Figure 4.1, the Analyzer, Detector, and Sanitizer, together with the domain profiles and transformation strategies they use.

Domain profiles. A profile collects the defaults for one domain: which entities count as sensitive, the prompts the agents use, and the detection and sanitization settings. Three profiles are provided: a generic baseline, a healthcare profile (clinical entities such as patient name, medical record number, dates, and insurance identifiers), and a humanitarian profile (data about affected populations such as names, location, ethnicity, and legal or displacement status).

Transformation strategies. A companion toolkit provides the four ways of removing sensitive spans, each available to the agents as a tool: masking with a placeholder label (`John Doe` becomes `[PERSON_1]`), replacement by a consistent fake identity (the same person maps to the same fake value within a request), the anonymization operators of Presidio, and a language-model rewrite that keeps the meaning of each passage while removing identifiers. The strategy comes from the active domain profile and can be overridden per run.

The three stages. The stages run in order, and each reads from and writes to a shared record for the request. The Analyzer selects the domain, by configuration or by inference from the query and the retrieved passages, and writes a policy: the entities to protect, the detection engine to use, and the sanitization strategy to apply. The Detector runs that engine on each passage, removes duplicate spans, and summarizes the risk. The Sanitizer applies the chosen strategy to those spans only; it performs no detection of its own, which keeps detection and transformation cleanly separate.

Illustration. Running one strategy on two short passages shows what sanitization produces. Listing 1 is the input and Listing 2 the result of placeholder masking; the other three strategies are shown in Appendix H.

```
[0] Patient John Doe (john.doe@example.com) was admitted on 2024-03-15 for review by Dr.
    Alice Smith.
[1] Follow-up call to John Doe at 555-123-4567, MRN AB12345678 confirmed.
```

Listing 1: Input chunks for the sanitization demo.

```
[0] Patient [PERSON_1] ([EMAIL_1]) was admitted on [DATE_1] for review by Dr. [PERSON_2].
[1] Follow-up call to [PERSON_1] at [PHONE_1]; MRN [MRN_1] confirmed.
```

Listing 2: Token masking output.

PR 4 – Adversarial agent and the pre-generation gate [GitHub]

Purpose. Check that the cleaned context is actually safe before it leaves the trusted environment (PR #338, issue #294, building on PR #337). Until this point the pipeline sanitized the context but never verified the result.

Approach. Three pieces are added. An adversarial agent probes the sanitized text for any identifier that survived, following a taxonomy of extraction, membership-inference, and reconstruction attacks [2]. When it reports a leak, a bounded loop makes the policy stricter, by lowering thresholds, adding engines, or choosing a stronger strategy, and cleans the context again, for a fixed number of attempts; if the context still cannot be cleared, the request is marked unsafe and no answer is produced. Finally, a human review gate shows a short summary that contains no sensitive information and waits for a person to approve, revise, or reject the context before it is used.

Design notes. Only genuine leaks count against the attempt limit, so a human reviewer can ask for as many revisions as needed. Because the summary shown at the gate carries no sensitive information, the review step itself cannot leak. The exact rule for tightening the policy at each step is a first version and may still change.

PR 5 – Integrate privacy into the RAG pipeline [GitHub]

Purpose. Complete the pipeline after the gate and connect it to MMORE's retrieval-augmented generation (PR #339, issue #295, building on PR #338). Until now the pipeline stopped at the gate without producing, checking, or recording an answer.

Approach. Once the gate approves, the answer model responds from the sanitized context alone and never sees the raw passages, so the privacy guarantee holds wherever that model runs. An advisory verifier then reads the answer and checks it for any residual leakage and for faithfulness to the retrieved evidence, raising a warning when needed but never cleaning the text itself. Every request ends with a short record that contains no sensitive information, which gives an auditable trail. The privacy pipeline replaces the ordinary answer step in MMORE's retrieval-augmented generation, so the report and any warnings are returned alongside the answer.

G Detection toolkit demo

This appendix reproduces in full the detection demo summarized in Appendix F (PR #335). The four engines were run on the synthetic note in Listing 3, at a confidence threshold of 0.4, with the following models: GLiNER (`nvidia/gliner-PII`); `openai/privacy-filter`; Microsoft Presidio with custom clinical recognizers; and an LLM engine using the small language model `Qwen/Qwen2.5-7B-Instruct` through DSPy. Tables G.1 to G.4 list every span each engine returned. For `openai/privacy-filter`, whose tagger emits sub-word BIO fragments, whitespace-only text is shown as ␣ and other leading or trailing spaces are omitted.

```
# Progress Note - Internal Medicine

Pt is a 58 yo M, goes by "Bobby," seen this AM on rounds (bed 4B, Tower 3).
Known to our service from the 3/2 admission - see prior note by Dr. Garcia.
Wife (Linda, reachable on her cell 617-555-0148, or at the house on Linwood
Ave) was at bedside overnight and is the HCP. Hx obtained partly from pt,
partly from the daughter who flew in from Austin.

Pt c/o "the same chest thing as before," denies fevers. Says he stopped the
metoprolol ~2 wks ago bc he "ran out and couldn't get through to the office."
Smokes ~1 ppd, quit date keeps changing.

Of note, records faxed over from Dr. R. Lee's office (St. Mary's, the one off
123 Main) list a different DOB than what we have - pt says 4/23/65 but the
face sheet says 04/23/1955, needs reconciling. MRN on the wristband (12345678)
matches the chart; the outside packet had 1245-6788 which is probably a
transcription error. Insurance still showing the old BCBS plan, member id
BCXY 99-88-77, though pt thinks he switched in Jan. Front desk left a vm at
555 867 5309 re: policy AB1234567.

Pt mentioned he emailed photos of the rash to "the dermatology guy" at
jsmith@hosp-derm.org last week, unclear which provider. Asked us to call his
brother (no name given, "he's a nurse over at the VA"). SSN partially visible
on a scanned form in the chart (xxx-xx-4321) - flagged to HIM.

A/P: 58M w/ CP, likely demand ischemia i/s/o med noncompliance. Will discuss
code status with pt and Linda. F/u cardiology (Dr. Maria Garcia, pager 12345)
after d/c. Tentative d/c 4/1, ride being arranged. Pt verbalized understanding,
quote: "just don't call me at work, my boss doesn't know."
```

Listing 3: Synthetic input note for the detection demo.

Table G.1: GLiNER (`nvidia/gliner-PII`). 15 spans at confidence threshold 0.4.

start	end	label	score	text
63	68	PERSON	0.859	Bobby
143	146	DATE	0.990	3/2
195	200	PERSON	0.990	Linda
224	236	PHONE	0.782	617-555-0148
381	387	LOCATION	0.942	Austin
640	650	LOCATION	0.987	St. Mary’s
723	730	DATE	0.959	4/23/65
755	765	DATE	0.974	04/23/1955
808	816	MRN	0.990	12345678
964	977	INSURANCE_ID	1.000	BCXY 99-88-77
1147	1167	EMAIL	1.000	jsmith@hosp-derm.org
1334	1345	SSN	0.963	xxx-xx-4321
1467	1472	PERSON	0.992	Linda
1494	1506	PERSON	0.637	Maria Garcia
1546	1549	DATE	0.789	4/1

Table G.2: `openai/privacy-filter`. 78 sub-word (BIO) spans at confidence threshold 0.4.

start	end	label	score	text
63	64	B-private_person	0.999	`B`
64	68	E-private_person	0.999	`obby`
176	179	B-private_person	1.000	`Dr`
179	180	I-private_person	1.000	`.`
180	187	E-private_person	1.000	`Garcia`
195	200	S-private_person	0.999	`Linda`
224	227	B-private_phone	1.000	`617`
227	228	I-private_phone	1.000	`-`
228	231	I-private_phone	1.000	`555`
231	232	I-private_phone	1.000	`-`
232	235	I-private_phone	1.000	`014`
235	236	E-private_phone	1.000	`8`
256	260	B-private_address	0.999	`Lin`
260	264	I-private_address	0.997	`wood`
264	265	I-private_address	0.999	`␣`
265	268	E-private_address	0.995	`Ave`
618	621	B-private_person	1.000	`Dr`
621	622	I-private_person	1.000	`.`
622	624	I-private_person	1.000	`R`
624	625	I-private_person	1.000	`.`
625	629	E-private_person	1.000	`Lee`
723	724	B-private_date	0.928	`4`
724	725	I-private_date	0.852	`/`
725	727	I-private_date	0.859	`23`
727	728	I-private_date	0.817	`/`
728	730	E-private_date	0.787	`65`
755	757	B-private_date	0.784	`04`
757	758	I-private_date	0.775	`/`
758	760	I-private_date	0.728	`23`
760	761	I-private_date	0.662	`/`
761	764	I-private_date	0.633	`195`
764	765	E-private_date	0.620	`5`
808	811	B-account_number	0.998	`123`
811	814	I-account_number	0.995	`456`
814	816	E-account_number	0.995	`78`
964	966	B-account_number	1.000	`BC`
966	968	I-account_number	0.999	`XY`
968	969	I-account_number	1.000	`␣`
969	971	I-account_number	0.999	`99`
971	972	I-account_number	0.999	`-`
972	974	I-account_number	0.999	`88`
974	975	I-account_number	0.997	`-`
975	977	E-account_number	0.999	`77`
1010	1014	S-private_person	0.494	`Jan`
1040	1043	B-private_phone	0.954	`555`
1043	1044	I-private_phone	0.894	`␣`
1044	1047	I-private_phone	0.880	`867`
1047	1048	I-private_phone	0.929	`␣`
1048	1051	I-private_phone	0.971	`530`
1051	1052	E-private_phone	0.947	`9`
1063	1066	B-account_number	0.992	`AB`
1066	1069	I-account_number	0.998	`123`
1069	1072	I-account_number	0.996	`456`
1072	1073	E-account_number	0.998	`7`
1147	1149	B-private_email	1.000	`js`
1149	1153	I-private_email	1.000	`mith`
1153	1154	I-private_email	1.000	`@`
1154	1155	I-private_email	1.000	`h`
1155	1158	I-private_email	1.000	`osp`
1158	1159	I-private_email	1.000	`-`
1159	1162	I-private_email	1.000	`der`
1162	1163	I-private_email	1.000	`m`
1163	1167	E-private_email	1.000	`.org`
1334	1337	B-account_number	0.977	`xxx`

start	end	label	score	text
1337	1338	I-account_number	0.991	-
1338	1340	I-account_number	0.950	xx
1340	1341	I-account_number	0.892	-
1341	1344	I-account_number	0.750	432
1344	1345	E-account_number	0.851	1
1466	1472	S-private_person	0.561	Linda
1490	1492	B-private_person	1.000	Dr
1492	1493	I-private_person	1.000	.
1493	1499	I-private_person	1.000	Maria
1499	1506	E-private_person	1.000	Garcia
1507	1513	B-account_number	0.993	pager
1513	1514	I-account_number	0.978	␣
1514	1517	I-account_number	0.945	123
1517	1519	E-account_number	0.978	45

Table G.3: Presidio with custom clinical recognizers. 26 spans at confidence threshold 0.4.

start	end	label	score	text
63	68	PERSON	0.850	Bobby
103	110	LOCATION	0.850	Tower 3
181	187	PERSON	0.850	Garcia
195	200	PERSON	0.850	Linda
224	236	PHONE_NUMBER	0.750	617-555-0148
381	387	LOCATION	0.850	Austin
480	487	DATE_TIME	0.850	wks ago
623	631	PERSON	0.850	R. Lee’s
640	650	PERSON	0.850	St. Mary’s
723	730	DATE_TIME	0.600	4/23/65
755	765	DATE_TIME	0.600	04/23/1955
755	765	HOSPITAL_DATE	0.600	04/23/1955
808	816	MRN	0.750	12345678
860	869	DATE_TIME	0.850	1245-6788
1011	1015	DATE_TIME	0.850	Jan.
1040	1052	PHONE_NUMBER	0.400	555 867 5309
1064	1073	INSURANCE_ID	1.000	AB1234567
1064	1073	US_DRIVER_LICENSE	0.650	AB1234567
1147	1167	EMAIL_ADDRESS	1.000	jsmith@hosp-derm.org
1154	1167	URL	0.500	hosp-derm.org
1168	1177	DATE_TIME	0.850	last week
1274	1276	LOCATION	0.850	VA
1467	1472	PERSON	0.850	Linda
1494	1506	PERSON	0.850	Maria Garcia
1514	1519	DATE_TIME	0.850	12345
1529	1544	PERSON	0.850	c. Tentative d/

Table G.4: LLM engine (`Qwen/Qwen2.5-7B-Instruct` via DSPy). 21 spans at confidence threshold 0.4.

start	end	label	score	text
63	68	PERSON	0.800	Bobby
143	146	DATE	0.800	3/2
177	187	PERSON	0.800	Dr. Garcia
195	200	PERSON	0.800	Linda
224	236	PHONE	0.950	617-555-0148
466	476	MEDICATION	0.800	metoprolol
478	483	DURATION	0.800	2 wks
619	629	PERSON	0.800	Dr. R. Lee
640	650	LOCATION	0.800	St. Mary’s
664	672	LOCATION	0.800	123 Main
808	816	MRN	0.950	12345678
860	869	MRN	0.950	1245-6788
964	977	INSURANCE_ID	0.950	BCXY 99-88-77

Table G.4: LLM engine (`Qwen/Qwen2.5-7B-Instruct` via DSPy). 21 spans at confidence threshold 0.4.

start	end	label	score	text
1040	1052	PHONE	0.950	`555 867 5309`
1147	1167	EMAIL	0.950	`jsmith@hosp-derm.org`
1274	1276	LOCATION	0.800	`VA`
1334	1345	SSN	0.950	`xxx-xx-4321`
1490	1506	PERSON	0.800	`Dr.  Maria Garcia`
1508	1519	PHONE	0.800	`pager 12345`
1546	1549	DATE	0.800	`4/1`
1631	1635	LOCATION	0.800	`work`

H Sanitization strategies demo

This appendix completes the sanitization demo of PR #337 summarized in Appendix F. Every strategy is applied to the input chunks of Listing 1 using the same hand-annotated spans (Table H.1), so the output reflects the strategy and not the detector. Token masking is shown in the main text (Listing 2); the remaining three strategies follow.

Table H.1: Hand-annotated spans for the sanitization demo.

chunk	label	score	text
0	PERSON	0.95	`John Doe`
0	EMAIL	0.95	`john.doe@example.com`
0	DATE	0.90	`2024-03-15`
0	PERSON	0.90	`Alice Smith`
1	PERSON	0.95	`John Doe`
1	PHONE	0.90	`555-123-4567`
1	MRN	0.95	`AB12345678`

```
[0] Patient David Pacheco (hlawrence@example.com) was admitted on 2016-02-12 for review by Dr.
     Karen Taylor.
[1] Follow-up call to David Pacheco at 775-592-9555x238; MRN MRN76684261 confirmed.
```

Listing 4: Entity replacement (Faker, `consistency=true`: `John Doe` maps to the same fake identity in both chunks).

```
operator=replace
[0] Patient <PERSON> (<EMAIL>) was admitted on <DATE> for review by Dr. <PERSON>.
[1] Follow-up call to <PERSON> at <PHONE>; MRN <MRN> confirmed.

operator=redact
[0] Patient  () was admitted on  for review by Dr. .
[1] Follow-up call to  at ; MRN  confirmed.

operator=mask  (masking_char='*', chars_to_mask=4)
[0] Patient **** Doe (****.doe@example.com) was admitted on ****-03-15 for review by Dr. ****
    e Smith.
[1] Follow-up call to **** Doe at ****123-4567; MRN ****345678 confirmed.

operator=hash
[0] Patient 243c714c...294a36db (a5c155fb...a69ee105) was admitted on bd203e8f...e4bb8098 for
     review by Dr. 8d28ccce...2f6c2f07.
[1] Follow-up call to a21921b8...52c683cba at 861e89f1...b0986720; MRN 64c4cab9...458acf27
    confirmed.
```

Listing 5: Presidio `AnonymizerEngine`, one block per operator.

```
[0] Patient (id: JOHNDOE) was admitted for review by Dr. (id: ALICESMITH).
[1] Follow-up call to PERSON at PHONE confirmed. MRN confirmed.
```

Listing 6: Synthetic rewrite (`Qwen/Qwen2.5-3B-Instruct`).